Road News — Documentação

Inteligência operacional para transporte rodoviário de cargas

Última atualização: 6 de agosto de 2026 às 16:47

Sumário

Parte I — Visão de Produto

1. Visão — Road News
2. Pitch — Road News
3. Modelo de Negócio — Road News
4. Pricing — Road News
5. Personas — Road News
6. Proposta de Valor — Road News
7. Mercado — Road News
8. Features — Road News
9. Concorrentes — Road News
10. FAQ — Road News
11. Onboarding — Road News
12. Material de Vendas — Road News

Parte II — Documentação Técnica

1. Road News Scraper
2. Arquitetura — Road News
3. Banco de Dados — Road News
4. API — Road News
5. Scrapers — Road News
6. Pipeline de IA — Road News
7. Deploy — Road News
8. Variáveis de Ambiente — Road News
9. Segurança — Road News
10. Contribuindo — Road News
11. Roadmap — Road News
12. Changelog — Road News
13. Arquitetura de Subagents — Road News
14. ADR 0001 — Arquitetura de Subagents do Road News

Changelog de publicação do PDF

Parte I — Visão de Produto

Visão — Road News

> Mantido por `product-docs-agent`. Última revisão: 2026-08-06 (primeira publicação).

Em uma frase

Fazer com que nenhuma gerenciadora de risco de transporte rodoviário de cargas seja pega de surpresa por um bloqueio, acidente ou evento crítico na rota de uma carga que ela está protegendo.

O problema que existe hoje

Notícia relevante para operação rodoviária está espalhada entre dezenas de portais, misturada com um volume muito maior de notícia irrelevante. Uma torre de controle ou gestor de risco que tentasse acompanhar isso manualmente teria que ler centenas de manchetes por dia para encontrar as poucas que de fato importam — e mesmo assim chegaria tarde, porque a notícia read não vem com "isso afeta a rota X, aqui está o desvio".

Onde o Road News quer chegar

Ser a camada de inteligência entre "aconteceu algo na estrada" e "a pessoa certa sabe, no tempo certo, o que fazer" — não mais um agregador de notícias, e sim um sistema que já filtrou, já classificou por gravidade, já cruzou com a rota real do cliente, e já sugere a ação.

Isso significa, na prática, evoluir em três direções ao mesmo tempo:

1. **Cobertura**: mais fontes, sem perder precisão — cada fonte nova passa por um filtro de relevância e legalidade antes de entrar.
2. **Velocidade**: o tempo entre "notícia publicada" e "cliente notificado" é o que separa um alerta útil de uma nota de rodapé.
3. **Confiança**: toda classificação de severidade vem com justificativa auditável — o cliente precisa poder confiar na régua, não só na resposta.

O que a visão não é

Não é virar um agregador de notícias genérico, nem um produto de mídia. Não é competir em volume de conteúdo — é competir em precisão do que chega até quem decide. Cada fonte, cada regra de classificação, cada alerta existe para responder a uma pergunta operacional concreta, nunca para "ter mais conteúdo".

Onde isso já é realidade, hoje

A base técnica da visão já existe em produção: coleta contínua, triagem de eventos críticos com evidência auditável, análise de rota cruzando trajeto real com notícia recente, e um modelo de acesso por pacote que já separa clientes por tenant com isolamento real de dado. O que falta não é a arquitetura — é ampliar cobertura de fonte, velocidade de alerta e canais de entrega (ver `FEATURES.md` e `../docs/ROADMAP.md`).

Pitch — Road News

> Mantido por `product-docs-agent`. Última revisão: 2026-08-06 (primeira publicação).

Elevator pitch (30 segundos)

Gerenciadoras de risco de transporte rodoviário de cargas perdem tempo lendo notícia genérica pra achar a que importa. O Road News monitora rodovias, clima e eventos logísticos em tempo real, filtra o que realmente afeta uma operação de carga, classifica por gravidade com justificativa auditável, e — na tela Minha Rota — cruza isso com o trajeto real de um veículo pra dizer se ele pode seguir, precisa desviar, ou precisa de validação antes de sair.

O problema

- Notícia relevante para rodovia está diluída entre um volume muito maior de notícia irrelevante, espalhada em dezenas de portais.
- Ler manualmente não escala e chega tarde — um bloqueio vale muito nos primeiros minutos e quase nada horas depois.
- Decisão de liberar ou não uma carga hoje depende de julgamento humano sobre informação fragmentada, sem trilha de auditoria de por que uma rota foi considerada segura.

A solução

Um pipeline que já resolve as três partes do problema:

1. **Coleta contínua** de fontes rodoviárias, climáticas e logísticas (nacionais e internacionais), agendada de hora em hora.
2. **Triagem por IA com evidência auditável** — todo evento crítico é classificado com confiança mínima e link para a fonte que sustenta a classificação, nunca uma alucinação.
3. **Cruzamento com a rota real** (Minha Rota): trajeto + trânsito real (Google Routes) + notícias das últimas 24h no raio da rota, com recomendação operacional final — e um caminho de fallback baseado em regra quando a IA não está disponível, pra nunca deixar a tela sem resposta.

Por que agora / por que este time

O produto já roda em produção com multi-tenancy real (isolamento de dado por cliente, papéis, pacotes com limite de uso), não é um protótipo — a base técnica que sustenta escalar cobertura e clientes já existe.

O que ainda não é verdade (honestidade deliberada)

Não há hoje alerta em tempo real fora do e-mail semanal, nem canal de WhatsApp/Telegram, nem app mobile nativo. Ver `../docs/ROADMAP.md` para o que é lacuna conhecida vs. o que é decisão deliberada de escopo.

Modelo de Negócio — Road News

> Mantido por `product-docs-agent`. Última revisão: 2026-08-06 (primeira publicação). Valores em R$ não fazem parte deste documento — ver `PRICING.md` para a estrutura de pacotes e a nota sobre validação pendente.

Formato

SaaS B2B, multi-tenant, por assinatura de pacote com limite de uso mensal — não é cobrança por notícia individual nem por API pública aberta. Cada cliente (`clients` no banco) tem um ou mais usuários (`profiles`), vinculados a um pacote (`packages`)

que define o que podem usar e quanto.

Como o pacote controla o que o cliente recebe

O modelo técnico já implementado (`api/_auth.js`) define, por pacote:

Dimensão	Controla
`monthly_news_limit`	Quantas consultas de notícia por mês
`monthly_ai_summary_limit`	Quantos resumos de IA (individual + semanal) por mês
`monthly_pdf_limit`	Quantos PDFs gerados por mês
`monthly_route_limit`	Quantas análises de rota (Minha Rota) por mês
`max_users`	Quantos usuários o cliente pode ter
`can_generate_individual_summary`	Se o resumo de IA por notícia está habilitado
`can_generate_weekly_summary`	Se o resumo semanal por e-mail está habilitado
`can_access_map`	Se o mapa rodoviário está habilitado

Papéis `admin`/`manager` (uso operacional interno da gerenciadora de risco) têm acesso ilimitado por desenho — a régua de pacote existe para o cliente final (`client`), não para quem opera a plataforma.

Quem vende, quem usa

Provisionamento hoje é manual: `admin`/`manager` criam o cliente, o(s) usuário(s), e vinculam o pacote pelo próprio painel (`/admin`, `/gestor`) — não há fluxo de self-signup. Ver `ONBOARDING.md`.

Custo variável principal

IA (OpenAI) é o maior custo variável direto por cliente/uso — por isso o produto já nasce com pré-filtro determinístico antes de qualquer chamada de IA e um interruptor de custo (`SCRAPER_AI_SUMMARIES_ENABLED`) desligado em produção até que o volume de clientes justifique religar (ver `finops-agent` em `../docs/AGENTS.md`).

O que este documento não define

Preço em R$ por pacote, número de tiers comerciais, e desconto por volume/contrato anual são decisões de negócio que exigem validação humana explícita — ver `PRICING.md`.

Pricing — Road News

> Mantido por `product-docs-agent`. Última revisão: 2026-08-06 (primeira publicação).
>
> **Nenhum valor em R$ está publicado aqui de propósito.** A regra deste projeto é clara: `product-docs-agent` nunca escreve preço final sem validação humana explícita — só estrutura a lógica de pacote/limite, que já existe implementada em código. Preencha os valores com o usuário antes de usar este documento externamente.

Estrutura técnica já implementada

Cada pacote (`packages` no banco) já define, tecnicamente, estas dimensões — a precificação real é decidir **quanto cada uma vale em R$** e **como combiná-las em tiers**:

Dimensão do pacote	O que limita
`monthly_news_limit`	Consultas de notícia por mês
`monthly_ai_summary_limit`	Resumos de IA (individual + semanal) por mês
`monthly_pdf_limit`	PDFs gerados por mês
`monthly_route_limit`	Análises de rota (Minha Rota) por mês
`max_users`	Usuários por cliente
`can_generate_individual_summary`	Liga/desliga resumo de IA por notícia
`can_generate_weekly_summary`	Liga/desliga resumo semanal
`can_access_map`	Liga/desliga acesso ao mapa

Hoje existe pelo menos um pacote ativo em produção (uso interno/piloto) — os valores exatos de limite desse pacote não estão neste documento porque são dado operacional, não posicionamento comercial; consultar diretamente a tabela `packages` via `database-expert/saas-platform-engineer` se necessário para uma proposta comercial concreta.

Perguntas que precisam de resposta humana antes de publicar preço

1. Quantos tiers comerciais existem (ex. Starter/Pro/Enterprise) e como cada um mapeia para as 8 dimensões acima?
2. Cobrança é por cliente (tenant) ou por usuário (`max_users`)?
3. Existe desconto por contrato anual/volume?
4. Minha Rota e o mapa são parte do pacote base ou add-on pago à parte?
5. Como funciona overage (cliente estoura o limite mensal antes do fim do mês) — bloqueia, cobra excedente, ou oferece upgrade automático? Hoje o comportamento técnico é **bloquear** com mensagem pedindo contato com o gestor (ver `FEATURE_RULES` em `api/_auth.js`) — decidir se isso é o comportamento comercial desejado ou só o piso técnico atual.

Quando este documento estiver validado

Ele deve conter: nome de cada tier, valor em R$/mês (ou critério de cobrança), o que cada tier inclui nas 8 dimensões acima, e a política de overage. Até lá, use `BUSINESS_MODEL.md` para entender a lógica sem os números.

Personas — Road News

> Mantido por `product-docs-agent`. Última revisão: 2026-08-06 (primeira publicação). Personas derivadas dos três papéis reais do sistema (`admin`/`manager`/`client` em `profiles.role`) e do domínio de gerenciamento de risco — não pesquisa de usuário formal ainda (ver nota no final).

Cliente operacional (papel `client`)

Quem é: gestor de risco ou analista de torre de controle numa transportadora, embarcador de carga de alto valor, ou na própria gerenciadora de risco atendendo um contrato de cliente.

O que precisa decidir: "esta carga pode sair agora por esta rota?" / "este alerta exige ação imediata ou é só monitoramento?"

Frustração que o produto resolve: informação sobre rodovia espalhada, sem gravidade clara, sem cruzamento com a rota real da carga.

Onde vive no produto: dashboard, alertas, mapa, Minha Rota, resumo semanal — as telas de consumo, limitadas pelo pacote do cliente.

O que faria essa persona abandonar o produto: alerta que não é acionável (severidade errada, sem justificativa), ou a tela ficar sem resposta quando a IA está fora do ar.

Gestor da plataforma (papel `manager`, aparece como "gestor" na UI)

Quem é: alguém dentro da gerenciadora de risco responsável por administrar os clientes que ela atende — criar cliente, vincular usuário, ajustar pacote, acompanhar consumo.

O que precisa decidir: "este cliente precisa de mais quota?" / "este usuário deveria ter acesso a quê?"

Onde vive no produto: telas `/gestor`, `/clientes`, `/usuarios`, `/pacotes`, `/consumo`, `/fontes-gestor`, `/relatorios-gestor`.

Acesso: operacional ilimitado (não é limitado por pacote como o `client`), mas não tem acesso administrativo total do sistema (isso é `admin`).

Administrador (papel `admin`)

Quem é: responsável técnico/operacional pela plataforma como um todo — não um cliente, não um gestor de conta.

Onde vive no produto: `/admin`, `/monitoramento`, `/integracoes`, `/banco`, `/logs`, e as telas `/developer/*` (visão mais técnica: fontes, relatórios, usuários, pacotes, consumo, configurações).

O que precisa decidir: saúde do sistema, integridade de dado, configuração de fonte/integração.

Nota sobre validação

Estas personas foram derivadas da estrutura técnica real (papéis, telas, limites de pacote) — não substituem entrevista com cliente real. Antes de usar em material de venda, validar com pelo menos um cliente/gestor real por persona.

Proposta de Valor — Road News

> Mantido por `product-docs-agent`. Última revisão: 2026-08-06 (primeira publicação).

A promessa central

Você não precisa ler notícia. Você precisa saber se a sua carga, na sua rota, agora, está segura para seguir — e o Road News entrega essa resposta, não uma lista de manchetes.

Por capacidade real (não aspiracional)

O cliente ganha	Como o produto entrega hoje
Não perder tempo filtrando notícia irrelevante	Coleta contínua já filtrada por keyword/relevância rodoviária antes de chegar à tela

O cliente ganha	Como o produto entrega hoje
Saber a gravidade real de um evento, não achismo	Classificação de risco (Baixo/Médio/Alto/Crítico) e, para eventos logísticos maiores, confiança mínima + evidência auditável (link da fonte)
Decidir sobre uma rota específica, não sobre o Brasil inteiro	Minha Rota: cruza o trajeto real (com trânsito, via Google Routes) com notícias das últimas 24h no raio da rota
Confiar na resposta mesmo quando a IA está fora	Toda análise crítica tem fallback determinístico por regra — a tela nunca fica sem resposta só porque a IA falhou
Ver o que aconteceu num mapa, não só numa lista	Mapa com marcador de risco, agrupamento por zoom, e trecho destacado quando há bloqueio/risco na rota
Resumo executivo sem precisar entrar no painel todo dia	Resumo operacional semanal por e-mail, gerado por IA
Confiança de que o dado do seu concorrente não aparece pro seu usuário	Isolamento de tenant real (RLS + validação em código), não um "modo demo" com dado compartilhado

Para quem isso importa mais

Gerenciadoras de risco e operações de torre de controle que decidem, em tempo real, se uma carga de alto valor pode seguir por uma rota — não para quem só quer "ficar informado" sobre rodovia em geral. Ver `PERSONAS.md`.

O que conscientemente não é a proposta de valor

Não é ser a fonte mais rápida de notícia genérica (existem agregadores maiores e mais rápidos para isso). Não é substituir o julgamento humano de quem libera a carga — é dar a esse julgamento a base de evidência que faltava. Não é cobertura internacional ampla — a cobertura internacional existe só onde afeta rota/porto/frete/combustível do Brasil (ver `../docs/AI_PIPELINE.md`, `STRATEGIC_INFRA_PATTERN`).

Mercado — Road News

> Mantido por `product-docs-agent`. Última revisão: 2026-08-06 (primeira publicação).
>
> **Aviso explícito**: este documento descreve o mercado qualitativamente. Números de tamanho de mercado (TAM/SAM/SOM em R$), quantidade de empresas endereçáveis, e penetração esperada **não estão aqui** porque exigiriam pesquisa de mercado real — publicá-los sem essa pesquisa seria inventar dado, o que viola a regra de não inflar posicionamento deste projeto. Pendente de validação humana antes de virar material de venda/investidor.

Categoria

Gerenciamento de risco em transporte rodoviário de cargas — um subconjunto de tecnologia de logística (logtech) focado especificamente em segurança e continuidade operacional de cargas de alto valor em trânsito, não em roteirização/TMS genérico nem em rastreamento de veículo por si só.

Por que este mercado existe no Brasil

O transporte rodoviário é o modal dominante de movimentação de carga no país — é fato amplamente conhecido do setor de logística brasileiro, não uma estatística proprietária deste documento. Isso cria demanda estrutural por: visibilidade de

risco em rodovia (acidente, bloqueio, clima severo, roubo de carga), continuidade em corredores estratégicos (acesso a portos, fronteiras), e resposta rápida a evento com potencial de interromper operação.

Quem compra hoje

- **Gerenciadoras de risco** que atendem transportadoras/embarcadores como serviço contratado — o Road News hoje é usado nesse modelo, com clientes cadastrados manualmente pela gerenciadora (ver `ONBOARDING.md`).
- **Transportadoras e embarcadores de carga de alto valor** — compradores potenciais diretos, ainda não validado como canal principal.

O que diferencia a categoria de "notícia genérica sobre rodovia"

Filtro por relevância operacional (não é "toda notícia de trânsito"), classificação de gravidade auditável, e cruzamento com rota real de uma carga específica — ver `VALUE_PROPOSITION.md` e `FEATURES.md`.

Próximo passo real para este documento

Antes de qualquer uso em pitch de investidor ou material de venda: (1) validar tamanho de mercado com fonte pública ou pesquisa paga (ex. associações do setor de transporte, CNT, ANTT), (2) confirmar com pelo menos 3 clientes/prospects reais se a categorização acima reflete como eles pensam o problema. `product-docs-agent` não substitui essa validação — só estrutura o documento pra ela ser preenchida.

Features — Road News

> Mantido por `product-docs-agent`. Última revisão: 2026-08-06 (primeira publicação). Cada feature abaixo existe de verdade no código (verificado contra `api/_auth.js: FEATURE_RULES` e as rotas de `vercel.json`) — nada aqui é aspiracional; itens futuros estão em `../docs/ROADMAP.md`.

Features controladas por pacote (o que pode ser limitado/vendido)

Feature	O que o cliente vê	Limite técnico que a sustenta
Consulta de notícias	Feed filtrado de notícias rodoviárias relevantes, atualizado de hora em hora	`monthly_news_limit`
Resumo individual por IA	Resumo de uma notícia específica, gerado por IA	`monthly_ai_summary_limit` + `can_generate_individual_summary`
Resumo semanal por IA	Resumo operacional da semana, por e-mail ou sob demanda no painel	`monthly_ai_summary_limit` + `can_generate_weekly_summary`
Geração de PDF	Exportar conteúdo em PDF	`monthly_pdf_limit`
Acesso ao mapa	Mapa rodoviário com marcadores de risco, agrupados por zoom	`can_access_map`
Minha Rota	Análise de trajeto: trânsito real + notícias cruzadas por proximidade + recomendação (IA com fallback por regra)	`monthly_route_limit`

Telas do painel (cliente)

Dashboard · Alertas · Central de Eventos · Notícias · Salvos · Resumo Operacional · Tendências · Fontes · Relatórios · Configurações · Mapa · Minha Rota · Resumo Semanal.

Telas de gestão (papéis `manager`/`admin`)

Admin · Gestor · Clientes · Usuários · Pacotes · Consumo · Fontes (gestor) · Relatórios (gestor) · Monitoramento · Integrações · Banco · Logs · e o conjunto `/developer/*` (visão técnica de clientes, fontes, relatórios, usuários, pacotes, consumo, configurações).

Como a classificação de gravidade funciona (o motivo pra confiar no alerta)

Toda notícia recebe um nível de risco (Baixo/Médio/Alto/Crítico). Eventos logísticos maiores (greve, bloqueio de porto, interdição estratégica) passam por uma triagem adicional com confiança mínima calculada e evidência (link da fonte) obrigatória — nunca uma classificação sem justificativa auditável. Ver `../docs/AI_PIPELINE.md`.

O que existe mas ainda não é "feature vendável" (uso interno/técnico)

Multi-tenancy, isolamento por RLS, papéis e provisionamento — são a fundação que sustenta tudo acima, não algo que o cliente final compra separadamente.

O que não existe ainda

Alerta em tempo real fora do e-mail semanal (WhatsApp/Telegram/webhook), app mobile nativo, self-signup sem intervenção de `manager`/`admin`. Ver `../docs/ROADMAP.md`.

Concorrentes — Road News

> Mantido por `product-docs-agent`. Última revisão: 2026-08-06 (primeira publicação).
>
> **Nenhum concorrente nomeado está listado aqui de propósito.** `product-docs-agent` não tem acesso a pesquisa de mercado real e nomear concorrentes específicos sem verificação seria inventar comparação — a regra deste projeto proíbe isso explicitamente. Este documento estrutura **como comparar**, não a comparação em si.

Categorias de alternativa que um prospect provavelmente já usa ou considera

1. **Leitura manual de notícia/redes sociais** — o "concorrente" mais comum na prática: alguém na operação acompanhando portais e grupos de WhatsApp de caminhoneiros manualmente. Não é uma empresa, mas é o que o Road News substitui na maioria dos casos hoje.
2. **Agregadores de notícia genéricos** (não especializados em rodovia/logística) — cobrem volume maior, mas sem filtro de relevância operacional nem classificação de gravidade.
3. **Plataformas de rastreamento/telemetria de frota (GPS/telemetria veicular)** — resolvem "onde está o veículo", não "o que está acontecendo na rota antes de eu decidir liberar a carga". Categoria adjacente, não necessariamente concorrente direto — potencial parceiro/integração.
4. **TMS (Transportation Management Systems)** com módulo de risco/monitoramento — se algum concorrente direto existir aqui, precisa ser identificado por pesquisa real, não suposição.
5. **Serviços de escolta/gerenciamento de risco tradicionais sem componente de software próprio** — o Road News pode ser ferramenta interna de uma gerenciadora de risco (nosso desenho atual) ou concorrer com quem oferece isso como parte de um serviço maior.

Eixos em que vale comparar (quando os nomes reais entrarem)

- Cobertura de fonte (quantidade e qualidade, não só quantidade).
- Velocidade entre publicação da notícia e alerta ao cliente.
- Se a classificação de gravidade vem com evidência auditável ou é caixa-preta.
- Se cruza notícia com a rota real do cliente ou só mostra notícia geral por região.
- Isolamento de dado por cliente (multi-tenant real) vs. painel compartilhado.
- Canais de alerta disponíveis (hoje o Road News só tem e-mail semanal — ponto real de desvantagem até `notification-engineer` ampliar, ver ../docs/ROADMAP.md).

Próximo passo real

Levantar 3–5 concorrentes/alternativas reais (nome, o que oferecem, preço se público) com o usuário ou com pesquisa dedicada, e preencher uma tabela comparativa nos eixos acima. Até lá, não usar este documento em material de venda além do que está aqui.

FAQ — Road News

> Mantido por `product-docs-agent`. Última revisão: 2026-08-06 (primeira publicação). Respostas verificadas contra o comportamento real do produto — nenhuma aqui descreve uma capacidade que não existe hoje.

De onde vêm as notícias? De portais e órgãos públicos monitorados continuamente (RSS, HTML, APIs públicas), incluindo INMET para clima e uma concessionária de rodovia para ocorrência em tempo real. Lista completa em ../docs/SCRAPERS.md.

Com que frequência o sistema atualiza? Coleta roda de hora em hora. A tela Minha Rota considera notícias das últimas 24h.

Como o sistema decide o que é "crítico"? Combinação de regra determinística (padrão de texto, infraestrutura estratégica) e classificação por IA com confiança mínima calculada — nunca uma classificação sem justificativa e evidência (link da fonte). Detalhe técnico em ../docs/AI_PIPELINE.md.

E se a IA estiver fora do ar? A tela Minha Rota continua funcionando com uma recomendação baseada em regra determinística (trânsito real + bloqueios detectados), sem depender da IA estar disponível.

Meus dados ficam visíveis para outros clientes? Não — isolamento de tenant por controle de acesso em duas camadas (banco de dados e código da API), não um "modo demo" compartilhado.

Como adiciono um usuário novo na minha empresa? Hoje é feito pelo gestor da plataforma (papel `manager/admin`), não por self-signup. Ver ONBOARDING.md.

Recebo alerta em tempo real? Hoje, o canal automatizado é o resumo semanal por e-mail (segunda-feira de manhã) — não há WhatsApp/Telegram/webhook ainda. Ver ../docs/ROADMAP.md.

Posso analisar uma rota específica antes de liberar uma carga? Sim — a tela Minha Rota cruza o trajeto (com trânsito real via Google Routes) com notícias recentes no raio da rota e devolve uma recomendação (seguir / atenção / validação obrigatória).

O que acontece se eu atingir o limite do meu pacote? A funcionalidade correspondente é bloqueada com uma mensagem específica, orientando contato com o gestor da plataforma para ajustar o pacote.

O produto cobre eventos fora do Brasil? Só quando afetam rota, porto, frete ou cadeia de suprimentos conectados ao Brasil (ex. fechamento de um canal estratégico internacional) — não é cobertura internacional geral.

Onboarding — Road News

> Mantido por `product-docs-agent`. Última revisão: 2026-08-06 (primeira publicação). Reflete o fluxo real de provisionamento hoje — manual, não self-service.

Fluxo real (hoje)

1. **Gestor da plataforma** (`manager/admin`) cria o cliente (`clients`) pelo painel — `/gestor` ou `/admin` → tela Clientes.
2. Cria o(s) usuário(s) (`profiles`) vinculados a esse cliente (`/usuarios`), com CPF, e-mail, telefone.
3. Vincula um pacote ativo (`/pacotes` → `user_packages`) definindo os limites mensais e as features habilitadas para aquele usuário/cliente.
4. Usuário recebe convite/acesso via Supabase Auth (template de convite em `scripts/supabase_invite_email_template.html`) e faz login.
5. Primeira experiência do usuário: dashboard, sem necessariamente nenhum alerta crítico ativo no momento — isso é esperado, não é um sinal de sistema quebrado (ver nota de design abaixo).

Não existe hoje

Self-signup, trial automático, ou onboarding sem intervenção de `manager/admin`. Todo cliente novo entra por provisionamento manual.

Nota de design importante

A maioria dos dias não tem evento crítico ativo — a tela principal precisa deixar claro que "sem alerta" é o sistema funcionando corretamente monitorando em segundo plano, não uma tela vazia por falha. Isso é responsabilidade de `ux-product-designer/frontend-engineer` ao desenhar o dashboard, mas é uma decisão de produto que nasce aqui: o valor do produto no dia a dia normal é a garantia silenciosa de monitoramento, não só o alerta em si.

O que falta para um onboarding self-service (se/quando fizer sentido)

Fluxo de cadastro próprio, seleção de pacote com pagamento integrado, e confirmação de e-mail sem intervenção manual — nenhum desses existe hoje; avaliar como iniciativa de produto separada, não assumir como próximo passo automático.

Material de Vendas — Road News

> Mantido por `product-docs-agent`. Última revisão: 2026-08-06 (primeira publicação). Ponto de partida para conversa comercial — não um script fechado, e não contém preço (ver `PRICING.md` sobre por quê).

Abertura (dor, não feature)

"Quanto tempo a sua operação gasta hoje tentando descobrir se uma rota está segura antes de liberar uma carga? E quando descobre, já é tarde?" — a conversa começa na dor de decisão sob informação fragmentada, não na lista de features.

Argumentos que já são verdade (usar com confiança)

- Classificação de gravidade **auditável**: todo evento crítico vem com evidência (link da fonte) e confiança mínima — não é "a IA disse". Isso importa especialmente em conversa com quem já foi pego de surpresa por um alerta impreciso antes.
- **Minha Rota** responde à pergunta específica que o comprador tem na cabeça: "esta carga, nesta rota, agora" — não "notícia de rodovia em geral".
- **Isolamento de dado real** entre clientes — argumento forte se o comprador já usou ferramenta com painel compartilhado ou "modo demo" disfarçado de multi-tenant.
- **Resiliência**: a análise de rota não trava se a IA cair — tem fallback determinístico. Bom argumento técnico para comprador com equipe de TI própria avaliando o produto.

Argumentos a NÃO usar (ainda não são verdade)

- Alerta em tempo real fora do e-mail semanal — **não existe hoje**. Não prometa.
- Cobertura internacional ampla — só existe onde afeta rota/porto/frete conectado ao Brasil.
- App mobile nativo — não existe.
- Comparação nomeada com concorrente específico — ver `COMPETITORS.md`, ainda sem pesquisa validada.

Objeções prováveis e como responder com honestidade

"Vocês têm alerta em tempo real?" — Hoje o canal automatizado é o resumo semanal por e-mail; a análise sob demanda (Minha Rota, resumo individual) é em tempo real dentro do painel. Alerta push/WhatsApp está no roadmap, não em produção.

"Como sei que a classificação de risco é confiável?" — Toda classificação crítica tem confiança mínima calculada e evidência rastreável (link da notícia original) — não é uma caixa-preta.

"E se a IA de vocês cair?" — A funcionalidade mais crítica (Minha Rota) tem fallback baseado em regra, não depende só da IA para dar uma resposta.

"Meu concorrente vai ver meus dados?" — Não; isolamento de tenant é arquitetural (banco + código), não uma promessa de política.

Próximo passo de venda hoje

Demo ao vivo da tela Minha Rota com uma rota real do prospect é o argumento mais forte disponível — mostra a proposta de valor central (rota + notícia + recomendação) sem depender de material comparativo ainda não validado.

Parte II — Documentação Técnica

Road News Scraper

Rotina de web scraping de **notícias rodoviárias** e **avisos meteorológicos** que afetam as estradas. Coleta de múltiplas fontes, filtra o conteúdo relevante e persiste em um banco **PostgreSQL (Supabase)**. Roda automaticamente a cada hora via **GitHub Actions** (`.github/workflows/scraper.yml`).

Documentação completa

Este README cobre o essencial para rodar o projeto. Para arquitetura, banco de dados, API, pipeline de IA e a arquitetura de subagents que mantém tudo isso atualizado, ver `CLAUDE.md` e `docs/AGENTS.md` — ou o PDF consolidado em `docs/dist/road-news-documentation.pdf`.

Fontes

11 fontes ativas hoje: RSS (G1, CNN Brasil, 4 buscas do Google News, Estradas.com.br, Blog do Caminhoneiro), HTML (Band), API JSON em tempo real (Arteris Fluminense) e avisos meteorológicos do INMET (filtrados por severidade, não por keyword). Lista completa e semântica exata do filtro de keyword em `docs/SCRAPERS.md`.

Tudo configurável em `config/sources.yaml`.

Como funciona

1. `main.py` lê as fontes de `config/sources.yaml`.
2. Cada fonte é coletada e filtrada (`scraper/sources/`).
3. Os itens são gravados no Postgres, deduplicados por URL (`scraper/storage/postgres.py`).

Ajustando o que é coletado (sem mexer no código) 👈

Tudo o que influencia os resultados fica em **`config/sources.yaml`** — é o "painel de controle" do projeto, todo comentado de forma didática:

- **`keywords`** — palavras que tornam uma notícia relevante (G1, CNN, Band). O scraper aplica as `keywords` apenas ao **título** da notícia. O casamento é por **palavra inteira** (`ferido` não casa com `ferimentos`); keywords com **várias palavras** (ex.: `incêndio em caminhão`) casam quando **todas** as palavras aparecem no título; e keywords com pontuação (ex.: `BR-`) casam por substring.
- **`severities` do INMET** — quais níveis de aviso de clima guardar (`Perigo`, `Grande Perigo`, etc.).
- **As fontes** (`rss_sources`, `html_sources`, `inmet_sources`) — quais sites/feeds são consultados.

Basta editar esse arquivo, salvar e dar `git push`: o GitHub Actions passa a usar a nova configuração na próxima execução. O motor que aplica as keywords fica em `scraper/utils/filters.py` (não precisa ser editado para ajustar a lista).

Rodar localmente

```
python -m venv .venv
.venv\Scripts\Activate.ps1      # Windows PowerShell
pip install -r requirements.txt
```

```
copy .env.example .env          # preencha DATABASE_URL
python main.py
```

Sem `DATABASE_URL` definido, o scraper apenas **exibe** os resultados no terminal (modo de teste, não grava no banco).

Execução agendada (GitHub Actions)

O workflow `.github/workflows/scraper.yml` roda a cada 2h. Configure o secret `DATABASE_URL` em **Settings** → **Secrets and variables** → **Actions** — use a connection string do *Session pooler* do Supabase (compatível com IPv4, exigido pelos runners do GitHub).

Weekly news com IA e e-mail

O workflow `.github/workflows/weekly-news.yml` gera um resumo semanal toda segunda-feira, 08:00 no horario de Brasilia, com a identidade visual Road News inspirada no padrão corporativo da Overhaul. Ele usa:

- `OPENAI_API_KEY` para montar o resumo operacional com IA.
- `RESEND_API_KEY` para enviar e-mail pelo Resend.
- `WEEKLY_EMAIL_TO` com os destinatarios separados por virgula.
- `WEEKLY_EMAIL_FROM` como remetente. Para producao, configure um dominio no Resend; para teste, o remetente padrao `onboarding@resend.dev` pode servir.
- `OPENAI_MODEL`, opcional, com padrao `gpt-4.1-mini`.

O front tambem tem o botao **Gerar resumo semanal**, que chama `/api/weekly-summary` no Vercel. Configure `OPENAI_API_KEY` e, se quiser, `OPENAI_MODEL` nas variaveis de ambiente do projeto Vercel.

Minha Rota com trânsito

A tela **Minha Rota** combina três fontes de informação:

- Google Routes API para distância, duração e condição de trânsito no traçado.
- Notícias das últimas 72 horas no Supabase, cruzadas por rodovia, localidade e proximidade geográfica do trajeto.
- OpenAI para produzir uma recomendação operacional baseada somente nos dados encontrados. Se a IA estiver indisponível, a tela mantém uma síntese segura baseada em regras.

Na Vercel, configure `GOOGLE_ROUTES_API_KEY` somente em **Environment Variables** para Production e Preview. A chave nunca é enviada ao navegador. No Google Cloud, habilite a **Routes API**, vincule billing e restrinja a chave para uso exclusivo dessa API.

Banco de dados

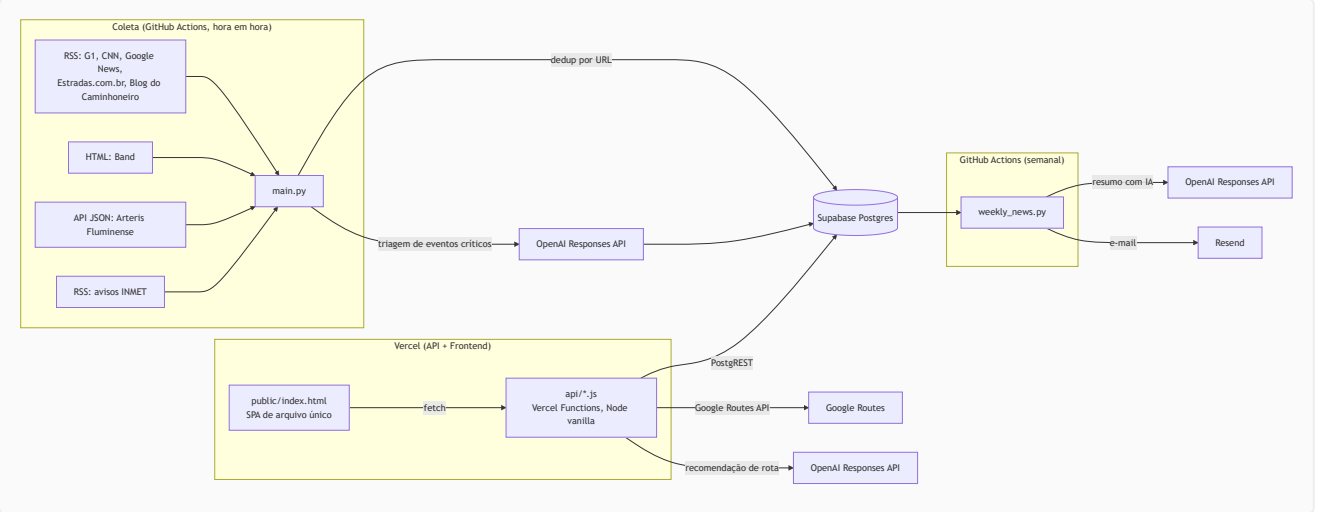
A tabela `news` é criada automaticamente na primeira execução, com constraint `UNIQUE(url)` para evitar duplicatas.

Arquitetura — Road News

> Mantido por `tech-docs-agent`. Reflete o estado real do código — não a stack de referência aspiracional (ver `CLAUDE.md` e `docs/adr/0001-arquitetura-de-subagents.md`).

Visão geral

Road News tem três componentes que rodam de forma desacoplada, sem framework de orquestração entre eles — cada um é implantado e escalado separadamente:



Os três componentes

1. Coleta (`main.py` + `scraper/`)

Python 3.11, agendado pelo GitHub Actions (`.github/workflows/scraper.yml`, `cron: "0 * * * *"` — de hora em hora, mais disparo em push na `main` e manual). Lê `config/sources.yaml`, roda cada fonte (`scraper/sources/*.py`), filtra por keyword (`scraper/utils/filters.py`), grava no Postgres com dedup por `UNIQUE(url)` (`scraper/storage/postgres.py`), e roda a triagem de eventos logísticos críticos (`scraper/event_intelligence.py`) quando `EVENT_INTELLIGENCE_ENABLED=true`. Sem browser headless — só `feedparser`/`requests`/`BeautifulSoup4`.

Dono: `scraping-engineer` (fontes/coleta) + `data-pipeline-engineer` (persistência/dedup/entidade) — ver docs/AGENTS.md.

2. API + Frontend (Vercel)

- **Frontend**: `public/index.html`, um único arquivo (~6.800 linhas) de HTML/CSS/JS vanilla — landing pública e painel logado (dashboard, alertas, mapa, Minha Rota, admin/gestor/developer) no mesmo arquivo, roteado no cliente via os `rewrites` de `vercel.json`. Sem build step, sem framework.
- **API**: `api/*.js`, Vercel Serverless Functions em Node.js puro (CommonJS), sem dependências npm, sem `package.json`. Cada arquivo exporta um `handler(req, res)`. Toda leitura/escrita de dado passa por chamadas REST diretas ao PostgREST do Supabase (`supabaseFetch` em `api/_auth.js`).

Dono: `frontend-engineer` (painel) + `backend-engineer` (endpoints de produto) + `saas-platform-engineer` (motor de auth/tenancy: `_auth.js`, `_admin.js`, `admin-*.js`).

3. Notificação semanal (`scraper/weekly_news.py`)

Roda toda segunda-feira 08:00 (America/Sao_Paulo) via `.github/workflows/weekly-news.yml`, gera um resumo operacional com IA (OpenAI) e envia por e-mail via Resend. Também disparável sob demanda pelo painel (`api/weekly-summary.js`) ou manualmente (`scripts/send_weekly_news.py`).

Dono: `notification-engineer` (entrega/template) + `ai-engineer` (chamada de IA) + `prompt-engineer` (texto do prompt).

Multi-tenancy

Já implementada em produção, não é um plano futuro: tabelas `clients`, `profiles` (papéis `admin`/`manager`/`client`), `packages`, `user_packages`, `usage_events`, `audit_logs`, com RLS via funções `SECURITY DEFINER` no Postgres. O controle de acesso e quota do lado da API é centralizado em `api/_auth.js` (`requireAccount`, `assertCanUse`, `recordUsage`, `FEATURE_RULES`). Ver `docs/DATABASE.md` e `docs/API.md`.

Por que não há framework/Docker/headless browser

Essas ausências são decisões, não lacunas: `system-architect` só introduz uma peça nova da "stack de referência" (Next.js/NestJS, Playwright, Docker) quando uma limitação real e observada justificar, registrada em ADR. Ver `docs/adr/` e o roster de agentes em `docs/AGENTS.md`.

Limites conhecidos do desenho atual

- `public/index.html` é um único arquivo grande — navegável via busca/grep, mas sem separação de módulo. Ver `frontend-engineer` em `docs/AGENTS.md` para o critério de quando isso justifica ser fatiado.
- A API não tem testes rodando em CI (só localmente) — ver `docs/CONTRIBUTING.md`.
- Extração de entidade (rodovia/km/UF) está hoje duplicada entre `scraper/utils/image_summary.py` (na coleta) e `api/news-feed.js` (na consulta) — consolidação é mandato aberto de `data-pipeline-engineer`.

Banco de Dados — Road News

> Mantido por `tech-docs-agent`. Dono do schema: `database-expert` (ver `docs/AGENTS.md`).

Postgres gerenciado pelo Supabase. **Não há framework de migration** — cada mudança de schema é um arquivo SQL idempotente em `scripts/`, aplicado manualmente no SQL editor do Supabase (`create ... if not exists`, `add column if not exists`, para poder rodar de novo com segurança).

Tabelas

news

Criada em runtime pelo próprio scraper (`scraper/storage/postgres.py`), não por um script em `scripts/`.

Coluna	Tipo	Observação
`id`	bigserial	PK
`title`, `url`, `source`, `category`, `published_at`, `summary`, `scraped_at`, `type`	text	Colunas base, sempre presentes
`url`	text	`UNIQUE` — é a chave de deduplicação primária
`created_at`	timestamptz	default `now()`
`image_summary`, `ai_summary`, `operational_relevance`, `operational_impact`, `ai_processed_at`, `ai_model`, `region`, `country`, `state`, `city`, `highway`, `risk_level`	—	**Opcionais**: adicionadas por migration posterior. O código consumidor (`api/news-feed.js`) tenta múltiplos conjuntos de colunas (`OPTIONAL_SETS`) para tolerar ambiente onde a migration ainda não rodou — nunca assuma que essas colunas existem sem checar.
`latitude`, `longitude`, `geo_precision`	—	Adicionadas por `scripts/supabase_geo_fields.sql`

RLS: `Allow public read news` (leitura pública, `anon+authenticated`) — a tabela de notícias em si não é segregada por tenant; o que é segregado por tenant é o **acesso** a ela via quota/pacote (ver `usage_events`).

`platform_events` + `event_intelligence_runs`

Criadas por `scraper/event_intelligence.py` em runtime (não por script em `scripts/`). `platform_events` guarda eventos logísticos críticos (`event_type = 'major_logistics_event'`), com `metadata` jsonb contendo `event_key`, `operational_status`, `history` (linha do tempo), `confidence`, `affected_infrastructure` etc. `event_intelligence_runs` é só controle de idempotência (fingerprint sha256 do lote analisado → não reanalisa o mesmo lote duas vezes). RLS habilitada; `event_intelligence_runs` é revogada de `anon`/`authenticated` (só acesso via service role).

Multi-tenancy (`scripts/supabase_production_clients_users_limits.sql` e relacionados)

Tabela	Papel
`clients`	Tenant. `company_name`, `status` (`active`/`inactive`/`blocked`).
`profiles`	Usuário. `role` (`admin`/`manager`/`client`), `client_id` (FK), `status`, `cpf` (mascarado em listagem), `phone`.
`packages`	Plano/pacote: `monthly_news_limit`, `monthly_ai_summary_limit`, `monthly_pdf_limit`, `monthly_route_limit`, `max_users`, `can_generate_individual_summary`, `can_generate_weekly_summary`, `can_access_map`, `is_active`.
`user_packages`	Vínculo usuário↔pacote ativo (`ends_at` nullable = sem expiração).
`usage_events`	Log de uso e de tentativa bloqueada (`event_type`, `client_id`, `user_id`, `package_id`, `tokens_input`/`tokens_output`, `metadata`). Fonte de dado para `data-analyst`.
`audit_logs`	Auditoria de ação administrativa.

RLS: todas com `enable row level security`. Autorização resolvida por funções `SECURITY DEFINER` (`is_admin`, `is_manager_or_admin`, `can_manage_target_user`, `get_user_role`) — nunca por policy inline duplicando essa lógica. Ver `scripts/supabase_production_clients_users_limits.sql` para o texto exato das policies.

Migrations existentes (`scripts/*.sql`)

Arquivo	O que faz
`supabase_auth_access.sql`	Base de auth/acesso
`supabase_packages_rls.sql`	RLS de pacotes
`supabase_platform_environments.sql`	Ambientes de plataforma
`supabase_production_clients_users_limits.sql`	Clientes/usuários/pacotes/limites (produção) — sem seed
`supabase_route_package_limit.sql`	Limite de análise de rota por pacote
`supabase_geo_fields.sql`	Campos de geolocalização em `news`
`supabase_image_summary_fields.sql`	Campos de enriquecimento por IA em `news`
`supabase_major_events.sql`	Suporte a `platform_events`
`supabase_secure_news_optional.sql`	Ajustes de segurança em colunas opcionais
`reactivate_admin_user.sql`	Script pontual de correção
`supabase_invite_email_template.html`	Template de e-mail de convite do Supabase Auth

Convenções para migration nova

1. Sempre idempotente (`if not exists`), documentada no cabeçalho do arquivo.
2. Nunca `DROP COLUMN`/`DROP TABLE` — o produto depende de histórico de notícias para tendência; prefira coluna nova + migração de leitura.
3. Toda tabela nova nasce com RLS habilitada e policy de negar-por-padrão.
4. Índice composto `(client_id, created_at desc)` obrigatório em tabela nova consultada por tenant.
5. Aplicação em produção é manual (SQL editor do Supabase) ou via `devops-engineer` — nunca automática a partir de um merge.

Ver `database-expert` em docs/AGENTS.md para o processo completo de proposta/revisão de schema.

API — Road News

> Mantido por `tech-docs-agent`. Donos do código: `backend-engineer` (endpoints de produto) e `saas-platform-engineer` (motor de auth/tenancy). Ver docs/AGENTS.md.

Todos os endpoints são Vercel Serverless Functions em `api/*.js`, Node.js puro (CommonJS, sem framework, sem dependência npm). Autenticação via Bearer token do Supabase Auth (`Authorization: Bearer <token>`); quando `AUTH_ENABLED=false`, a API roda em modo demo sem exigir login (nunca libera Admin nesse modo).

Convenção de acesso

Praticamente todo endpoint de produto começa chamando `assertCanUse(req, "<feature>")` (definido em `api/_auth.js`) e termina registrando uso com `recordUsage`. Isso resolve, nessa ordem: usuário autenticado → perfil → cliente (tenant) → pacote ativo → papel (`admin`/`manager`/`client`) → checagem de feature flag/limite mensal do pacote. `admin`/`manager` (`isPrivileged`) têm acesso operacional ilimitado por desenho.

Erros voltam como `{ "error": "mensagem específica" }`, nunca stack trace ou detalhe interno do Supabase.

Endpoints de produto (dono: `backend-engineer`)

Endpoint	Métodos	Feature/quota	Descrição
`/api/news-feed`	`GET`, `POST`	`news_query` (GET) / `route_analysis` + `ai_summary_individual` (POST)	`GET`: feed de notícias (últimas, INMET, recentes 24h, mapa 30d, eventos maiores). `POST`: análise da tela **Minha Rota** (Google Routes + notícias cruzadas por rodovia/proximidade + recomendação por IA com fallback determinístico).
`/api/news-summary`	`POST`	`ai_summary_individual`	Resumo de uma notícia específica via IA.
`/api/weekly-summary`	`GET`, `POST`	`ai_summary_weekly`	Geração/consulta do resumo semanal (o mesmo fluxo do e-mail, disparável pelo painel).
`/api/my-access`	`GET`	—	Retorna pacote ativo, papel e limites do usuário autenticado (usado pelo painel para exibir consumo).
`/api/usage-event`	`POST`	—	Registro de evento de uso genérico vindo do frontend.
`/api/auth-config`	`GET`	—	Configuração pública de auth (se está habilitada, URL/chave pública do Supabase).

Endpoints de plataforma / backoffice (dono: `saas-platform-engineer`)

Endpoint	Métodos	Papel exigido	Descrição
`/api/admin-data`	`GET`, `POST`, `PATCH`	admin/manager	Dados agregados do backoffice (dashboards internos).
`/api/admin-packages`	`POST`, `PATCH`, `DELETE`	admin/manager	CRUD de pacotes (limites, flags).
`/api/admin-user-package`	`POST`	admin/manager	Vincula pacote a usuário.
`/api/admin-user-profile`	`GET`, `PATCH`	admin/manager	Consulta/edita perfil de usuário (CPF mascarado na listagem).
`/api/admin-user-status`	`PATCH`	admin/manager	Ativa/inativa usuário.
`/api/admin-users`	`POST`, `DELETE`	admin/manager	Cria/remove usuário.

`api/_auth.js` e `api/_admin.js` não são endpoints — são o módulo compartilhado que todos os arquivos acima importam (prefixo _ = não roteável pela Vercel).

Erros comuns e seus significados

Situação	Mensagem típica
Sessão ausente/expirada	"Login necessário." / "Sessão inválida ou expirada."
Sem pacote vinculado	"Vincule um pacote para liberar os recursos da plataforma."
Feature desabilitada no pacote	Mensagem específica por feature (ex. "Resumo semanal não está disponível no seu pacote.")
Limite mensal atingido	Mensagem específica por feature (ex. "Limite mensal de consultas do Minha Rota atingido…")
`GOOGLE_ROUTES_API_KEY` ausente/inválida	"A análise com trânsito ainda não está configurada…" / "A chave da API de rotas não foi aceita…"
Acesso negado (papel insuficiente)	"Acesso negado."

Ver o objeto `FEATURE_RULES` em `api/_auth.js` para a lista completa e exata de mensagens por feature.

O que não existe hoje

Sem versionamento de API (`/api/v1/...`), sem OpenAPI/Swagger, sem rate limiting explícito além da quota por pacote, sem paginação cursor-based (os endpoints usam `limit` fixo por chamada). Essas são lacunas conhecidas, não bugs — avaliar com `system-architect` antes de assumir que qualquer uma delas precisa ser resolvida.

Scrapers — Road News

> Mantido por `tech-docs-agent`. Dono do código: `scraping-engineer` (coleta) + `data-pipeline-engineer` (persistência). Ver docs/AGENTS.md.

Como funciona

1. `main.py` lê `config/sources.yaml`.
2. Cada fonte é coletada por um módulo de `scraper/sources/`: `rss.py` (feeds RSS/Atom via `feedparser`), `html.py` (raspagem HTML via `requests+BeautifulSoup4`, hoje só a Band), `inmet.py` (avisos meteorológicos, filtrados por severidade, não por keyword), `arteris.py` (API JSON pública da concessionária Arteris Fluminense, tempo real).
3. Itens passam pelo motor de keyword (`scraper/utils/filters.py`) antes de entrar na lista final — exceto fontes que já são inerentemente relevantes (INMET, Arteris, que não usam keyword).
4. `scraper/storage/postgres.py` grava no Postgres, deduplicando por `UNIQUE(url)`.
5. `scraper/event_intelligence.py` roda a triagem de eventos críticos sobre o lote coletado (ver `docs/AI_PIPELINE.md`).

Sem browser headless em nenhuma etapa — nenhuma fonte hoje exige renderização JS para ser lida.

Fontes ativas (`config/sources.yaml`)

Fonte	Tipo	Categoria	Observação
G1 - Brasil	RSS	geral	Filtrado por keyword
CNN Brasil	RSS	geral	Filtrado por keyword
Google News — Incidentes em rodovia	RSS (busca)	rodovia	Query já filtra por rodovia+incidente
Google News — Caminhão/carreta	RSS (busca)	rodovia	Query já filtra por veículo+incidente
Estradas.com.br	RSS	rodovia	Portal especializado
Blog do Caminhoneiro	RSS	transporte	Portal especializado
Google News — Eventos logísticos Brasil	RSS (busca)	evento logístico	`max_items: 30`, keywords próprias (greve, bloqueio, porto…)
Google News — Corredores logísticos internacionais	RSS (busca)	logística internacional	`max_items: 30`, cobre Ormuz, Suez, Mar Vermelho, Panamá
Band - Notícias	HTML	geral	Único scraper HTML puro hoje
Arteris Fluminense (tempo real)	API JSON	rodovia	Sem keyword — toda ocorrência é relevante; dedup por hash de conteúdo (sem URL por ocorrência)
INMET - Avisos Meteorológicos	RSS	clima	Filtro por severidade (`Perigo`, `Grande Perigo`, `Perigo Potencial`) + tipo de evento opcional

Semântica do matching de keyword (`scraper/utils/filters.py`)

Aplicado só a **título e subtítulo/resumo** (nunca ao texto completo do artigo):

- Case-insensitive.
- Palavra inteira: `ferido` **não** casa com `ferimentos`.
- Keyword com múltiplas palavras (ex. `"incêndio em caminhão"`): casa quando **todas** aparecem no título, em qualquer posição — não exige frase exata.
- Keyword com pontuação (ex. `"BR-"`): casa por substring.

Mudar essa semântica afeta recall retroativamente — qualquer alteração aqui precisa de aviso a `data-analyst` (para medir o impacto) e atualização do comentário em `config/sources.yaml`.

Atribuição de fonte nas queries “Google News”

As 4 fontes `Google News – ...` são buscas agregadas (`news.google.com/rss/search?q=...`), não veículos editoriais. Até esta revisão, `scraper/sources/rss.py` gravava o campo `source` da notícia com o nome literal da query (ex. `"Google News - Caminhão/carreta"`), o que aparecia no card do cliente como se fosse a fonte da notícia — problema de credibilidade sinalizado por `compliance-advisor`.

Correção: `scrape_rss()` agora resolve o veículo real por trás do redirect, nesta ordem:

1. `entry.source.title` do feed do Google News (quando o feed expõe o elemento `<source>`).
2. Fallback: sufixo `"Título - Veículo"` do próprio `entry.title` (padrão do Google News quando `<source>` não vem preenchido).
3. Só cai de volta no nome da query (`"Google News - ..."`) se nenhum dos dois acima existir.

Isso só se aplica às fontes cujo `name` em `config/sources.yaml` começa com `"Google News"` — as demais fontes (G1, CNN Brasil, Estradas.com.br, etc.) já gravam o próprio nome do veículo.

Adicionar uma fonte nova

Sem editar código Python: adicione um bloco em `rss_sources`/`html_sources`/`inmet_sources` de `config/sources.yaml`, salve, `git push` — o GitHub Actions usa a versão do repositório na próxima execução (até 1h de defasagem). Editar código só é necessário para um **tipo** de fonte novo (ex. um segundo scraper HTML) — nesse caso, o `type` em `html_sources` precisa de um raspador correspondente em `scraper/sources/html.py`.

Ordem de preferência ao integrar uma fonte nova (do mais barato ao mais caro de manter): **RSS/feed > API JSON pública > HTML estático > renderização client-side** (headless browser — hoje inexistente no projeto, exige ADR de `system-architect` para introduzir).

Toda fonte nova passa por `compliance-advisor` (robots.txt/ToS) **antes** de ser ativada em produção.

Agendamento

`.github/workflows/scraper.yml`: cron `0 * * * *` (hora em hora, UTC) + disparo em push na `main` + `workflow_dispatch` manual. Timeout de 15 minutos. Roda com `EVENT_INTELLIGENCE_ENABLED=true` e `SCRAPER_AI_SUMMARIES_ENABLED=false` (enriquecimento de IA por item desligado por custo — ver `docs/AI_PIPELINE.md`). Em push, também roda `scripts/upsert_santos_trucker_event.py` (evento curado do Porto de Santos).

> Nota de divergência corrigida: versões anteriores da documentação mencionavam execução “a cada 2h” — o cron real (`scraper.yml`) é de hora em hora.

Pipeline de IA — Road News

> Mantido por `tech-docs-agent`. Donos do código: `ai-engineer` (infraestrutura de chamada), `prompt-engineer` (texto de prompt), `logistics-intelligence` (taxonomia/schema/régua). Ver `docs/AGENTS.md`.

Todo uso de IA no Road News é via **OpenAI Responses API** (`gpt-4.1-mini` como modelo padrão, configurável por variável de ambiente). Não há roteamento multi-provedor (Anthropic/Google) — é caixa de ferramentas futura.

Os 4 pontos de uso

1. Triagem de eventos logísticos críticos (`scraper/event_intelligence.py`)

O padrão-ouro do projeto: roda a cada execução do scraper (se `EVENT_INTELLIGENCE_ENABLED=true`), sobre o lote recém-coletado.

- **Pré-filtro determinístico** (regex) antes de qualquer chamada de IA: `candidate_reasons()` combina padrões de contexto logístico, gravidade, impacto operacional, infraestrutura estratégica e resolução — só o que casa é enviado à IA. Notícia com padrão de "greve não-logística" (professores, bancários…) é descartada mesmo antes do pré-filtro positivo, a menos que também case com impacto logístico.
- **Schema estrito** (`EVENT_RESPONSE_SCHEMA`, `strict: true`): `is_major_event`, `event_key`, `category`, `scope` (`nacional`/`internacional`), `severity` (sempre `critical` neste pipeline), `status` (`em_andamento`/`monitoramento`/`encerrado`), `confidence`, `evidence` (com URL/fonte/data rastreável).
- **Confidence mínimo**: `0.82` (`MIN_CONFIDENCE`) — abaixo disso, descartado.
- **Idempotência**: fingerprint sha256 do lote de candidatos + modelo; lote já analisado não gera nova chamada (`event_intelligence_runs`).
- **Anti-alucinação**: o prompt exige evidência real (URL/título/fonte copiados das notícias fornecidas), proíbe inventar local/desvio/duração/encerramento, e cada evento validado (`_valid_events`) precisa de `evidence` não-vazia e `affected_infrastructure` preenchida.
- Grava/atualiza em `platform_events`, preservando `event_key` entre execuções (mesma ocorrência não vira evento novo) e mantendo `history` (linha do tempo de mudança de status).

2. Enriquecimento por item (`scraper/utils/image_summary.py`)

Roda por notícia individual na coleta, **desligado em produção hoje** (`SCRAPER_AI_SUMMARIES_ENABLED=false` no cron — decisão de custo, ver `finops-agent`). Quando habilitado, preenche `ai_summary`, `operational_relevance`, `operational_impact`, `region`/`country`/`state`/`city`/`highway`, `risk_level`. Tem fallback determinístico (`build_image_summary_fallback`) para quando a IA está desabilitada ou indisponível.

2.1. Resumo do botão "Gerar Resumo" (`api/news-summary.js` → `generateAiSummary`)

Gera o resumo individual mostrado no card compartilhável (canvas em `public/index.html`, `drawShareImage`). Campos obrigatórios do JSON: `image_summary`, `ai_summary`, `region`, `country`, `state`, `city`, `highway`, `km`, `direction`, `status`, `category`, `risk_level`, `operational_relevance`, `operational_impact`, `insufficient_data`.

- **Regra de corte mínimo**: quando não há rodovia/km/UF identificável nem categoria operacional clara, o modelo marca `insufficient_data: true` em vez de gerar uma frase vaga tipo "impacto ainda não confirmado" sem dizer o motivo — o frontend decide como exibir isso, não é responsabilidade do texto "empurrar" a incerteza.
- `km`, `direction` (sentido) e `status` (ativo/controlado/encerrado) vêm vazios (`""`) quando a fonte não informa — nunca inventados.
- `fallbackSummary()` (sem `OPENAI_API_KEY`) preenche a mesma forma de dado, com `km`/`direction`/`status` sempre vazios (a extração determinística por regex ainda não infere esses três campos).
- O card de imagem (`drawShareImage`) agora renderiza um badge de rota (rodovia + região) além dos badges de risco/relevância, e a cor do badge de risco varia por severidade (crítico/alto/médio/baixo) em vez de usar sempre a cor de marca.

3. Resumo semanal por e-mail (`scraper/weekly_news.py`)

Gera o resumo operacional enviado toda segunda-feira (ver `docs/DEPLOY.md` para o agendamento). Também acionável sob demanda via `api/weekly-summary.js`.

4. Recomendação da tela "Minha Rota" (`api/news-feed.js` → `generateRouteAnalysis`)

Combina trajeto real (Google Routes API, com trânsito) + notícias das últimas 24h cruzadas por rodovia/proximidade geográfica + recomendação por IA. **Tem fallback determinístico completo** (`fallbackRouteAnalysis`): se a chave de API

estiver ausente/inválida ou a chamada falhar, a tela continua funcionando com uma síntese baseada em regras (status `validacao_obrigatoria/atencao/sem_restricao_confirmada` calculado a partir de bloqueios/tráfego detectados, sem IA).

Padrões que qualquer chamada de IA nova precisa seguir

1. Pré-filtro determinístico antes da chamada (regra/heurística que elimina o óbvio sem custo).
2. `strict: true` no JSON schema — nunca parsing best-effort de texto livre para dado que vai para produção.
3. Fallback funcional sem a chave de API presente ou com a chamada falhando.
4. Evidência rastreável — nunca gravar campo que a IA "alucinou" sem URL/fonte real por trás.
5. Modelo sempre configurável por variável de ambiente (`OPENAI_MODEL`, `OPENAI_EVENT_MODEL`, `OPENAI_IMAGE_MODEL`), nunca hardcoded sem override.
6. Idempotência: reprocessar o mesmo lote não pode gerar uma segunda chamada nem duplicar resultado.

Ver `ai-engineer` em `docs/AGENTS.md` para o processo completo de revisão dessas regras.

Custo

Ver `finops-agent` — o controle de custo mais importante hoje é justamente o enriquecimento por item estar desligado em produção, e o pré-filtro determinístico de `event_intelligence.py` reduzir o volume que chega a custar chamada de IA.

Deploy — Road News

> Mantido por `tech-docs-agent`. Dono: `devops-engineer`. Ver `docs/AGENTS.md` e `../VERCEL_DEPLOY.md` para o passo a passo original de setup na Vercel.

Infraestrutura real: **só Vercel + GitHub Actions**. Sem Docker, Coolify, Railway ou Azure.

Vercel (frontend + API)

- Serve `public/**` como site estático e `api/*.js` como Serverless Functions.
- `vercel.json` define `cleanUrls`, `trailingSlash: false`, e os `rewrites` que mapeiam as rotas do painel (`/dashboard`, `/mapa`, `/minha-rota`, `/admin`, `/gestor`, `/developer/*` etc.) de volta para `/` — o roteamento real é client-side, dentro de `public/index.html`.
- Deploy dispara em push na branch configurada como produção (normalmente `main`).
- Variáveis de ambiente de produção configuradas em **Vercel → Project Settings → Environment Variables** (nunca em `.env` commitado) — ver `docs/ENVIRONMENT.md` para a lista completa e onde cada uma vai.

GitHub Actions (coleta + notificação)

Workflow	Gatilho	O que faz
`.github/workflows/scraper.yml`	`cron: "0 * * * *"` (hora em hora) + push na `main` + manual	Roda `python main.py`; em push, também roda `scripts/upsert_santos_trucker_event.py`. Timeout 15 min.
`.github/workflows/weekly-news.yml`	Segunda-feira 08:00 (America/Sao_Paulo) + manual	Gera e envia o resumo semanal por e-mail.

Segredos configurados em **Settings → Secrets and variables → Actions** do repositório GitHub (`DATABASE_URL`, `OPENAI_API_KEY` no mínimo).

`DATABASE_URL` **precisa** usar a connection string do *Session pooler* do Supabase (compatível com IPv4, exigido pelos runners do GitHub) — não a connection string direta.

Rollback

Sem pipeline de rollback automatizado hoje. Na prática:

- **Vercel**: reverter para um deployment anterior pelo painel da Vercel (cada push gera um deployment imutável) ou `git revert` + novo push.
- **GitHub Actions**: `workflow_dispatch` permite rodar manualmente após reverter o código; não há "desfazer" uma execução já concluída (ex. um evento crítico já publicado precisa ser corrigido via UPDATE manual ou nova execução que o reclassifique).
- **Banco**: migrations são desenhadas para nunca remover dado (ver `docs/DATABASE.md`) — rollback estrutural real é manual, com o DBA (`database-expert`) revisando antes.

Observabilidade

Logs de execução agendada: aba **Actions** do GitHub (por run). Logs de função serverless: painel da Vercel (**Deployments** → **Functions**). Não há dashboard de erro/latência centralizado (ex. Sentry/Datadog) hoje — lacuna conhecida, não avaliada ainda por `system-architect`.

Antes de propor uma mudança de infraestrutura

Docker, fila gerenciada, serviço novo, ou qualquer coisa além de Vercel + GitHub Actions exige ADR de `system-architect`, motivado por uma limitação real observada (timeout de função, necessidade de processo de longa duração etc.) — nunca por preferência de arquitetura.

Variáveis de Ambiente — Road News

> Mantido por `tech-docs-agent`, sempre em sincronia com `.env.example` (que `devops-engineer` mantém). Se este documento e `.env.example` divergirem, `.env.example` é a fonte de verdade — atualize aqui para bater.

Variável	Usada por	Onde configurar	Descrição
`DATABASE_URL`	scraper (`main.py`)	Local (`.env`) / GitHub Secret	Connection string Postgres do Supabase. Em GitHub Actions, **precisa** ser a do *Session pooler* (IPv4). Sem ela, o scraper só exibe resultado no terminal (modo teste).
`OPENAI_API_KEY`	scraper + API	Local / GitHub Secret / Vercel Env Var	Chave da OpenAI, usada em todos os 4 pontos de IA (ver `docs/AI_PIPELINE.md`).
`OPENAI_MODEL`	scraper + API	Opcional, qualquer ambiente	Modelo default para resumo semanal e recomendação de rota. Padrão: `gpt-4.1-mini`.
`OPENAI_IMAGE_MODEL`	scraper (`image_summary.py`)	Opcional	Modelo para enriquecimento por item (hoje desligado em produção via `SCRAPER_AI_SUMMARIES_ENABLED`).

Variável	Usada por	Onde configurar	Descrição
`OPENAI_EVENT_API_KEY`	scraper (`event_intelligence.py`)	Opcional	Override de chave só para a triagem de eventos críticos; se ausente, usa `OPENAI_API_KEY`.
`OPENAI_EVENT_MODEL`	scraper (`event_intelligence.py`)	Opcional	Override de modelo só para a triagem de eventos críticos.
`EVENT_INTELLIGENCE_ENABLED`	scraper	GitHub Actions (`scraper.yml`)	`"true"` em produção — liga a triagem de eventos críticos.
`SCRAPER_AI_SUMMARIES_ENABLED`	scraper	GitHub Actions (`scraper.yml`)	`"false"` em produção — enriquecimento de IA por item desligado por custo.
`GOOGLE_ROUTES_API_KEY`	API (`news-feed.js`)	**Somente Vercel** Environment Variables	Chave da Google Routes API (tela Minha Rota). Habilitar a Routes API no Google Cloud, vincular billing, restringir a chave a essa API. **Nunca** enviada ao navegador.
`GOOGLE_MAPS_API_KEY`	API (`news-feed.js`)	Vercel	Fallback aceito se `GOOGLE_ROUTES_API_KEY` não estiver setada.
`AUTH_ENABLED`	API	Vercel	`true` (padrão/produção) exige login e valida cotas. `false` só para preview/demo — nunca libera Admin no modo demo. Vazio em produção = falha fechado (exige login).
`NEXT_PUBLIC_AUTH_ENABLED`	API	Vercel	Mantido só por compatibilidade com deploys antigos; equivalente a `AUTH_ENABLED`.
`SUPABASE_URL`	API	Vercel (público)	URL do projeto Supabase.
`SUPABASE_ANON_KEY`	API	Vercel (público)	Chave pública/anon do Supabase — usada também pelo navegador.
`SUPABASE_SERVICE_ROLE_KEY`	API	**Somente Vercel**, nunca no navegador	Chave de service role — acesso privilegiado ao Supabase. Vazamento é incidente de segurança imediato.
`RESEND_API_KEY`	notificação (`weekly_news.py`)	Local / GitHub Secret / Vercel	Chave do Resend para envio do resumo semanal.
`WEEKLY_EMAIL_FROM`	notificação	Qualquer ambiente	Remetente do e-mail semanal. Padrão de teste: `onboarding@resend.dev`; produção exige domínio configurado no Resend.
`WEEKLY_EMAIL_TO`	notificação	Qualquer ambiente	Destinatários, separados por vírgula.

Regras

- `.env` nunca é commitado (`.gitignore` já cobre `.env*`, exceto `.env.example`).
- Chaves de service role/privilegiadas (`SUPABASE_SERVICE_ROLE_KEY`) só existem no lado servidor (`api/**`, scraper) — nunca em `public/**`.
- Toda variável nova é adicionada a `.env.example` (por `devops-engineer`) **e** a esta tabela (por `tech-docs-agent`) no mesmo PR — nunca só uma das duas.

Segurança — Road News

> Mantido por `tech-docs-agent`. Dono da revisão contínua: `security-reviewer` (somente leitura — acha e prioriza, não corrige). Ver docs/AGENTS.md.

Modelo de autenticação e autorização

- **Autenticação**: Supabase Auth, Bearer token (`Authorization: Bearer <token>`) — não é sessão por cookie, então CSRF clássico é baixo risco (ver `security-reviewer`).
- **Autorização**: resolvida sempre no servidor a partir do `user_id` autenticado (`api/_auth.js: requireAccount`), nunca de um campo enviado pelo cliente. Papel (`admin/manager/client`) e pacote/quota controlam o que cada requisição pode fazer.
- **Isolamento de tenant**: duas camadas — RLS no Postgres (via funções `SECURITY DEFINER`) **e** filtro explícito por `client_id`/`user_id` no código da API (defesa em profundidade; RLS é a rede de segurança, não a única validação).

Superfícies de risco reais deste produto

1. **SSRF** — o backend busca URLs a partir de fontes de scraper, Google Routes, OpenAI. Qualquer URL nova buscada pelo servidor a partir de dado não confiável precisa de allowlist de domínio/esquema.
2. **XSS armazenado** — título/resumo de notícia vêm de fontes externas não confiáveis e são renderizados em `public/index.html` via `innerHTML`. O projeto já tem um helper `escapeHtml` (baseado em `textContent`, seguro) usado consistentemente nos pontos revisados — toda interpolação nova de campo de origem externa em `innerHTML` precisa passar por ele.
3. **Vazamento de tenant** — o risco mais caro do produto: um cliente ver dado de outro. Mitigado por RLS + filtro explícito, testado com usuário de papel/tenant diferente antes de qualquer mudança em `api/_auth.js`, `api/admin-*.js` ou policy de RLS.
4. **Exposição de segredo** — `SUPABASE_SERVICE_ROLE_KEY`, `OPENAI_API_KEY`, `DATABASE_URL`, `RESEND_API_KEY`. Nenhuma dessas chaves aparece em `public/**` nem em log de execução (GitHub Actions loga host/porta da conexão, nunca a connection string inteira).
5. **LGPD** — duas frentes distintas: dado do usuário da plataforma (CPF mascarado em listagem, `api/_admin.js`) e dado pessoal de terceiro citado em notícia raspada (nome de vítima de acidente, por exemplo) — segunda frente é responsabilidade de `compliance-advisor`, não do código.

O que NÃO é prioridade de revisão aqui

CSRF (modelo Bearer-token, não cookie) — não gaste ciclo de revisão nisso a menos que um fluxo novo introduza cookie de sessão.

Processo

Toda mudança em `api/_auth.js`, `api/admin-*.js`, políticas de RLS, ou qualquer URL nova buscada pelo servidor passa por `security-reviewer` antes do merge — achados classificados `[BLOQUEANTE]/[ALTO]/[MÉDIO]/[SUGESTÃO]`, cada um com arquivo:linha e correção proposta (não implementada por ele). Achado `[BLOQUEANTE]` impede merge.

Se um segredo vazar

Rotação de credencial é decisão humana imediata, fora do fluxo normal de revisão — `security-reviewer` escala diretamente ao usuário ao encontrar evidência de vazamento em produção (não só no código-fonte).

Reportar uma vulnerabilidade

Este é um projeto interno; reporte diretamente à equipe responsável pelo repositório, sem abrir issue pública descrevendo o achado antes de uma correção estar disponível.

Contribuindo — Road News

> Mantido por `tech-docs-agent`.

Rodando localmente

Scraper (Python 3.11)

```
python -m venv .venv
.venv\Scripts\Activate.ps1      # Windows PowerShell
pip install -r requirements.txt
copy .env.example .env          # preencha DATABASE_URL (ver docs/ENVIRONMENT.md)
python main.py
```

Sem `DATABASE_URL`, o scraper só exibe os resultados no terminal (não grava no banco) — bom para testar fonte nova sem risco.

API + Frontend

Sem build step. Para testar `public/**` localmente, sirva a pasta com qualquer servidor estático (ex. `python -m http.server` dentro de `public/`) ou use `vercel dev` se a CLI da Vercel estiver instalada. Os endpoints de `api/**` dependem das variáveis de ambiente do Supabase/OpenAI/Google Routes estarem configuradas (ver docs/ENVIRONMENT.md).

Publicador de documentação (Node.js)

```
cd scripts
npm install      # baixa o Chrome do Puppeteer no postinstall — primeira vez demora mais
npm run build:docs-pdf
```

Gera/atualiza `docs/dist/road-news-documentation.pdf`. Ver `docs-publisher` em docs/AGENTS.md.

Testes

Sem framework externo — bibliotecas nativas, coerente com a filosofia zero-dependência do projeto:

```
python -m unittest discover tests    # Python: event_intelligence, curated_event, image_summary,
summary_generation, timezone
node --test tests/                   # JS: test_production_access.js
```

Nenhum workflow do GitHub Actions roda essa suíte automaticamente hoje — rodar é manual, antes de propor merge. Isso é uma lacuna conhecida (ver docs/ROADMAP.md), não uma escolha deliberada.

Regra de teste: para bug reportado, o teste vermelho vem antes da correção. Para fonte de scraper nova, use fixture de HTML/RSS congelado, nunca a URL ao vivo. Nenhum teste usa credencial de produção nem dado real de cliente.

Arquitetura de agentes

Este projeto usa 23 subagents especializados do Claude Code (`.claude/agents/*.md`). Antes de propor uma mudança maior, leia `CLAUDE.md` (mapa de roteamento rápido) e `docs/AGENTS.md` (roster completo, matriz de propriedade de arquivo, workflows). Regra central: **um dono por arquivo** — se não estiver claro quem deveria mexer em algo, é sinal de arbitrar com `system-architect`, não escolher um dono ad hoc.

Definition of Done

Nenhuma mudança está pronta sem, nesta ordem: `tech-docs-agent` (toda mudança técnica) → `product-docs-agent` (se mudou capacidade percebida pelo cliente) → `project-historian` (sempre, registra em `docs/HISTORY.md`) → `docs-publisher` (regenera o PDF se `/product` ou `/docs` mudou). Gate antes do merge: `qa-test-engineer` verde → `security-reviewer` sem achado `[BLOQUEANTE]`.

Convenções de código

- Scraper: Python 3.11, sem browser headless. Prefira RSS/API pública a HTML scraping ao adicionar fonte.
- API: Node.js puro, sem framework, sem dependência npm nova sem aprovação de `system-architect` (exceção já aprovada: `scripts/` do `docs-publisher`, isolado do resto do projeto).
- Frontend: HTML/CSS/JS vanilla em `public/index.html`, sem bundler.
- SQL: migration sempre idempotente, nunca `DROP COLUMN`/`DROP TABLE`.
- Commits: mensagem descreve o comportamento observável da mudança, não só o arquivo tocado (ver `git log` para o padrão já usado no projeto: `feat:`, `fix:`, `security:`, `data-quality:`).

Roadmap — Road News

> Mantido por `tech-docs-agent`. Lista lacunas reais e conhecidas do estado atual — não promessas de prazo. Cada item vira trabalho real quando um agente (ou o usuário) o priorizar explicitamente; nada aqui está em execução só por estar listado.

Confiabilidade / Qualidade

- **CI não roda a suíte de testes automaticamente.** `tests/` existe e passa localmente, mas nenhum workflow do GitHub Actions executa `unittest`/`node --test` hoje. Dono: `devops-engineer` + `qa-test-engineer`.
- **Sem observabilidade centralizada** (erro/latência) além dos logs nativos de GitHub Actions e Vercel. Avaliar necessidade real antes de introduzir uma ferramenta nova (ver `system-architect`).
- **Extração de entidade duplicada** entre `scraper/utils/image_summary.py` (na coleta) e `api/news-feed.js` (na consulta). Mandato aberto de `data-pipeline-engineer`.

Produto

- **`/product` ainda não tem conteúdo publicado formalmente** além do primeiro corte gerado por `product-docs-agent` — pricing em R$ e nomes de concorrentes específicos seguem pendentes de validação humana (ver `docs/../product/PRICING.md` e `product/COMPETITORS.md`).
- **Notificação em tempo real** (WhatsApp, Telegram, webhook) não existe — hoje só o resumo semanal por e-mail. Dono: `notification-engineer`, condicionado a demanda real e a análise de custo de `finops-agent`.
- **`docs/domain/`, `docs/ux/`, `docs/metrics/`, `docs/seo/`** existem como pastas mas ainda sem conteúdo — cada uma é mandato do respectivo agente (`logistics-intelligence`, `ux-product-designer`, `data-analyst`, `seo-content-agent`).

Infraestrutura / IA

- **Roteamento multi-provedor de IA** (Anthropic/Google além de OpenAI) — caixa de ferramentas futura, só entra via ADR de `system-architect` quando custo ou qualidade justificarem.
- **pgvector / busca semântica** — não avaliado ainda; hoje a busca é por keyword determinística.
- **Enriquecimento de IA por item** (`SCRAPER_AI_SUMMARIES_ENABLED`) está desligado em produção por custo — religar exige aprovação de `finops-agent` com projeção de custo mensal.
- **Headless browser (Playwright/Crawl4AI/Firecrawl)** — nenhuma fonte hoje exige renderização JS; só entra se uma fonte concreta justificar, via ADR.

Fora do escopo por decisão, não por esquecimento

Docker/Coolify/Railway/Azure, framework web (Next.js/NestJS), bundler de frontend, versionamento de API — todos deliberadamente fora até que uma limitação real do produto os justifique. Ver docs/adr/0001-arquitetura-de-subagents.md.

Changelog — Road News

> Mantido por `tech-docs-agent`, uma entrada por mudança de **comportamento observável** (não por refactor interno sem efeito externo). Não confundir com o changelog interno do PDF em `docs/dist/.docs-pdf-manifest.json` (esse é só sobre a publicação do PDF em si) nem com docs/HISTORY.md (registro fiel de toda mudança, incluindo internas, mantido por `project-historian`).

2026-08-06

- **Adicionado**: arquitetura de 23 subagents especializados do Claude Code (`.claude/agents/`), documentada em `docs/AGENTS.md` e `CLAUDE.md`.
- **Adicionado**: `docs-publisher` — consolida `/product` + `/docs` em um PDF único e sempre atualizado (`docs/dist/road-news-documentation.pdf`).
- **Corrigido**: resumo do card gerado pelo botão "Gerar Resumo" (canvas em `public/index.html`) podia truncar sem indicação visual de corte (`completeCardSummary`); agora sempre termina com reticências quando o texto é cortado.
- **Melhorado**: card do botão "Gerar Resumo" ganhou badge de rota (rodovia + região) e cor de severidade específica no badge de risco (crítico/alto/médio/baixo), em vez de usar a cor de marca para todos os níveis.
- **Melhorado**: prompt do resumo individual (`api/news-summary.js`) passa a extrair km, sentido e status da via quando existirem, e marca `insufficient_data: true` explicitamente em vez de gerar frase vaga de impacto não confirmado.
- **Corrigido**: fontes "Google News — …" (`config/sources.yaml`) gravavam o nome da query de busca como fonte da notícia; `scraper/sources/rss.py` agora resolve o veículo real por trás do redirect.

2026-08-05

- **Melhorado**: marcadores de risco sobrepostos no mapa agora se agrupam e reagrupam por nível de zoom.
- **Corrigido**: rodovias citadas na análise de rota sem geolocalização precisa agora ainda recebem marcador no mapa.
- **Melhorado**: marcador da rota mostra km e descrição curta sempre visíveis.
- **Melhorado**: visibilidade de trechos de risco no mapa da tela Minha Rota.
- **Adicionado**: sinalização de bloqueios/riscos na rota com descrição e segmento destacado no mapa.
- **Corrigido**: janela de notícias consideradas na análise de rota reduzida de 72h para 24h (relevância).
- **Adicionado**: suporte a paradas intermediárias na análise de rota (Minha Rota).
- **Corrigido**: lacunas de classificação de risco identificadas em QA visual; bug de KPI exibindo vazio quando o valor real era zero.
- **Melhorado**: resumos de IA agora são instruídos a agregar valor além de repetir o título.

- **Corrigido**: campo de região corrompido quando a saída de IA não era validada.
- **Melhorado**: classificação de risco diferenciada em vez de default para "Alto".
- **Segurança**: removidos detalhes internos de arquitetura do copy do painel admin.

Entradas anteriores a 2026-08-05 não foram retroativamente documentadas aqui — consultar `git log` para o histórico completo do repositório.

Arquitetura de Subagents — Road News

Este documento é o guia de uso, para humanos, do roster de 23 subagents do Road News. Os agentes em si vivem como arquivos executáveis em `.claude/agents/*.md` — o frontmatter (`description`) é o que o Claude Code usa para rotear automaticamente; este documento explica quando cada um é acionado, o que cada um nunca faz, como se relacionam, e os workflows que orquestram vários agentes numa mudança real. O racional da decisão está em `docs/adr/0001-arquitetura-de-subagents.md`.

Premissa estrutural, válida para todo este documento: os agentes descrevem e defendem o stack real do Road News hoje — scraper Python sem browser headless (feedparser/BeautifulSoup), API em Vercel Serverless Functions com Node vanilla sem dependências (sem package.json), frontend em arquivo único (public/index.html, sem build step), Supabase Postgres com RLS, deploy só Vercel + GitHub Actions, testes com `unittest`/`node:test`. A stack "de referência" mais ampla (Next.js/NestJS, Playwright/Crawl4AI/Firecrawl, Docker/Coolify/Railway/Azure, Jest/Vitest) é uma **caixa de ferramentas futura**: `system-architect` e o agente dono de cada artefato a habilitam sob demanda, via ADR, quando um caso concreto justificar — nunca por padrão.

Roster

Camada A — Arquitetura e Implementação

Slug	Modelo	Missão em uma linha
`system-architect`	opus	Estrutura, limites de módulo, ADRs, avaliação de impacto arquitetural
`scraping-engineer`	sonnet	Coleta: fontes RSS/HTML/API, seletores, keywords, resiliência
`data-pipeline-engineer`	sonnet	Normalização, deduplicação semântica, resolução de entidade
`ai-engineer`	sonnet	Infraestrutura de chamada a LLM: schema, fallback, cache, custo por chamada
`prompt-engineer`	sonnet	Texto de prompt, versionamento, teste A/B contra dataset dourado
`logistics-intelligence`	opus	Taxonomia, régua de severidade, formato de inteligência operacional
`database-expert`	opus	Schema, migrations, índices, RLS, planos de execução
`backend-engineer`	sonnet	Endpoints de produto em `api/**`
`frontend-engineer`	sonnet	Painel em `public/index.html`
`notification-engineer`	sonnet	Entrega de alerta/resumo: e-mail, roteamento, anti-flood
`saas-platform-engineer`	sonnet	Multi-tenancy: auth, papéis, pacotes, quotas, provisionamento
`devops-engineer`	sonnet	CI/CD, GitHub Actions, Vercel, variáveis de ambiente
`qa-test-engineer`	sonnet	Testes unitários, regressão, contrato

Camada B — Governança e Revisão (somente leitura de código)

Slug	Modelo	Missão em uma linha
`security-reviewer`	opus	SSRF, XSS, isolamento de tenant, segredo exposto, authz/authn
`compliance-advisor`	opus	robots.txt/ToS, direito autoral, LGPD de dado citado em notícia
`finops-agent`	opus	Custo total (IA, infra, terceiros) e trade-off de economia

Camada C — Produto e Dados

Slug	Modelo	Missão em uma linha
`ux-product-designer`	sonnet	Fluxo e hierarquia de informação do painel
`data-analyst`	sonnet	Métricas de volume, qualidade, latência, engajamento
`seo-content-agent`	sonnet	Meta tags, robots.txt/sitemap.xml, palavra-chave

Camada D — Documentação e Memória Institucional

Slug	Modelo	Missão em uma linha
`product-docs-agent`	sonnet	`/product`: posicionamento, pricing, personas, mercado
`tech-docs-agent`	sonnet	README + `/docs`: arquitetura, API, banco, deploy
`project-historian`	haiku	`docs/HISTORY.md`: registro fiel de toda mudança
`docs-publisher`	haiku	Consolida `/product` + `/docs` em um PDF único, sempre no mesmo caminho

---

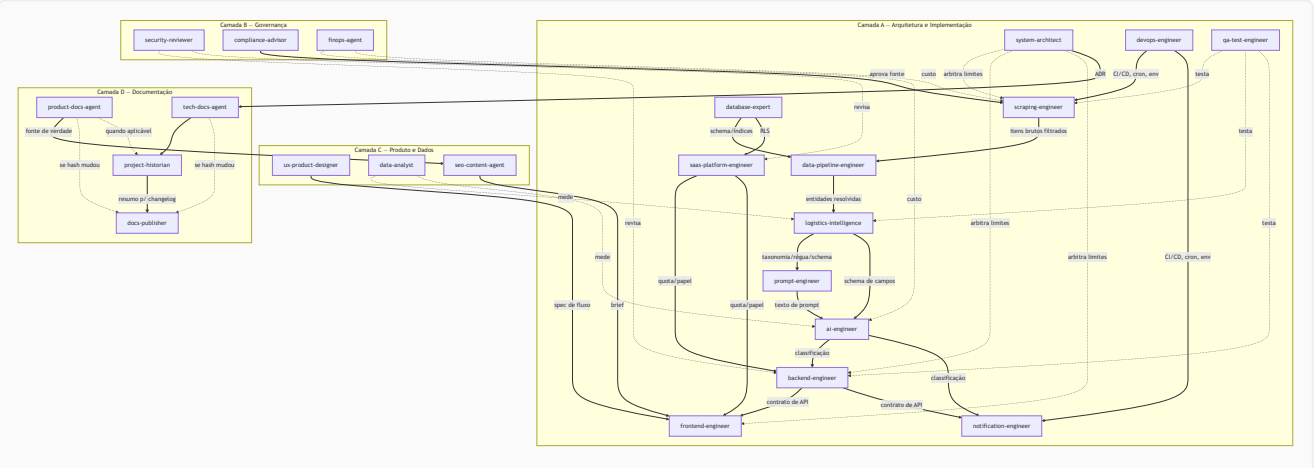
Matriz de propriedade de arquivo

Princípio inegociável: nenhum caminho abaixo tem dois agentes com `Write/Edit`.

Caminho	Dono exclusivo
`scraper/sources/**`, `scraper/utils/filters.py`, `config/sources.yaml`	scraping-engineer
`scraper/storage/**`, resolução de entidade/dedup semântica	data-pipeline-engineer
Código de chamada a LLM dentro de `event_intelligence.py`, `image_summary.py`, `weekly_news.py`, `news-feed.js`	ai-engineer
`/prompts/**` (texto de prompt, a criar)	prompt-engineer
`docs/domain/**` (taxonomia, régua, dataset dourado)	logistics-intelligence
`scripts/*.sql`	database-expert
`api/news-feed.js`, `news-summary.js`, `weekly-summary.js` (orquestração), `my-access.js`, `usage-event.js`, `auth-config.js`	backend-engineer
`public/**` exceto `robots.txt/sitemap.xml/blog/**`	frontend-engineer
`scraper/weekly_news.py` (template/envio), `api/weekly-summary.js` (disparo), `scripts/send_weekly_news.py`, conteúdo/destinatários de `.github/workflows/weekly-news.yml`	notification-engineer

Caminho	Dono exclusivo
`api/_auth.js`, `api/_admin.js`, `api/admin-*.js` (6 arquivos)	saas-platform-engineer
`.github/workflows/**` (mecânica: cron, steps, secrets), `vercel.json`, `.vercelignore`, `.env.example`	devops-engineer
`tests/**`	qa-test-engineer
`docs/adr/**`	system-architect
`public/robots.txt`, `public/sitemap.xml`, futuro `public/blog/**`, `docs/seo/**`	seo-content-agent
`docs/ux/**`	ux-product-designer
`docs/metrics/**`	data-analyst
`/product/**`	product-docs-agent
`README.md`, `docs/*.md` (raiz), `docs/diagramas/**`, `docs/AGENTS.md`	tech-docs-agent
`docs/HISTORY.md`	project-historian
`docs/dist/road-news-documentation.pdf`, `docs/dist/.docs-pdf-manifest.json`, `scripts/build-docs-pdf.js`, `scripts/package.json`	docs-publisher
— (sem propriedade de arquivo, só leitura)	security-reviewer, compliance-advisor, finops-agent

Grafo de dependências



Sobreposições resolvidas

Herdadas da especificação original do roster

1. **`news-intelligence-agent` + `news-intelligence-analyst` → fundidos em `logistics-intelligence`.** Ambos operavam sobre a mesma notícia com o mesmo conhecimento de domínio — mantê-los separados criaria duas fontes de verdade para a taxonomia.
2. **`ai-engineer` × `prompt-engineer` — mantidos separados.** Código (roteador, cache, schema, retries) e texto (prompt versionado, testado contra dataset dourado) têm ciclos de teste e critério de sucesso distintos.

3. **`cost-optimization-agent` → `finops-agent`, escopo ampliado.** Restrito a LLM seria subconjunto de `ai-engineer`; ampliado a custo total (infra, storage, terceiros) é transversal a A4/A7/A12 e legitimamente separado.
4. **`documentation-writer` = `tech-docs-agent`.** Duplicata eliminada.
5. **`product-docs-agent` × `seo-content-agent` — hierarquia.** `/product` é fonte de verdade de posicionamento; SEO deriva dela.
6. **`security-reviewer` × `compliance-advisor` — perguntas diferentes.** "Dá pra invadir/vazar?" vs. "podemos legalmente coletar, armazenar e redistribuir isso?" exigem conhecimento totalmente distinto.
7. **`data-analyst` × `ai-engineer`/`prompt-engineer`.** Quem mede não deve ter incentivo de melhorar o próprio número.

Encontradas durante a implementação (residuais desta entrega)

8. **ADR: `system-architect` vs. `tech-docs-agent`.** A especificação original listava ADRs como responsabilidade dos dois (A1 "escreve ADRs"; D2 "mantém /docs e /docs/adr"). Resolvido: `system-architect` é o único com `Write` em `docs/adr/**` — é quem decide e assina a decisão arquitetural. `tech-docs-agent` lê e referencia ADRs a partir de `docs/*.md`, nunca os edita.
9. **`docs/adr` também aparecia na missão de `project-historian`.** A especificação original listava "Registra… em `/docs/HISTORY.md` + `/docs/adr`" para D3. Resolvido: `project-historian` só escreve em `docs/HISTORY.md`; quando a mudança nasceu de uma decisão arquitetural, ele **referencia** o número do ADR, nunca escreve dentro de `docs/adr/**`.
10. **`api/**` dividido entre `backend-engineer` e `saas-platform-engineer`.** A especificação original dava "APIs REST" a A8 e "multi-tenancy" a A11 sem dividir arquivos concretamente — no repositório real isso colidiria em todo `api/*.js`. Resolvido: `saas-platform-engineer` é dono exclusivo do motor de tenancy (`api/_auth.js`, `api/_admin.js`, `api/admin-*.js`, 6 arquivos); `backend-engineer` é dono dos endpoints de produto que **consomem** esse motor (`news-feed.js`, `news-summary.js`, `weekly-summary.js`, `my-access.js`, `usage-event.js`, `auth-config.js`).
11. **`scraper/storage` entre `scraping-engineer` e `data-pipeline-engineer`.** Resolvido: `scraping-engineer` é dono da coleta e do filtro de keyword (`scraper/sources/**`, `scraper/utils/filters.py` — que o próprio README do projeto já trata como parte da coleta); `data-pipeline-engineer` é dono da persistência e da resolução de entidade (`scraper/storage/**`).
12. **`scraper/weekly_news.py` toca três agentes ao mesmo tempo.** É um único arquivo que gera texto com IA, usa um prompt, e envia e-mail. Resolvido por sub-responsabilidade dentro do mesmo arquivo: chamada de IA = `ai-engineer`; texto do prompt = `prompt-engineer`; template/destinatário/disparo = `notification-engineer`. Nenhum edita a parte dos outros sem coordenação explícita — é a única exceção deliberada a "um dono por arquivo", justificada por ser um arquivo pequeno com responsabilidades genuinamente ortogonais em faixas de linha diferentes.
13. **Carve-out de `/docs` para C1/C2/C3.** A especificação original não detalhava onde `ux-product-designer`, `data-analyst` e `seo-content-agent` escrevem, o que colidiria com `tech-docs-agent`. Resolvido: subpastas próprias — `docs/ux/`, `docs/metrics/`, `docs/seo/`.
14. **`public/index.html` entre `frontend-engineer` e `seo-content-agent`.** A landing pública e o painel logado são o mesmo arquivo (SPA de arquivo único) — um cenário que a especificação original, pensada para uma landing separada de Next.js, não previa. Resolvido: `frontend-engineer` mantém `Write` exclusivo sobre `public/index.html`; `seo-content-agent` entrega brief/diff textual de meta tag e copy, nunca edita o arquivo diretamente. `seo-content-agent` só tem `Write` direto sobre artefatos que genuinamente não colidem (`public/robots.txt`, `public/sitemap.xml`, futuro `public/blog/**`).
15. **`docs-publisher` (adicionado depois do roster inicial de 22) introduziu a primeira dependência npm do projeto.** `scripts/build-docs-pdf.js` usa `markdown-it` + `puppeteer`, isolados num `scripts/package.json` próprio — não afeta `api/` (que continua zero-dependência) nem o scraper Python. Essa decisão foi validada explicitamente com o usuário antes da implementação, diante da alternativa de escrever em Python (mais consistente com o resto do projeto, mas não executável na máquina de desenvolvimento usada na sessão, que não tinha Python instalado). Registrada aqui, não em ADR à parte, por ser uma decisão pontual de ferramenta de build, não uma mudança de arquitetura de produto.

16. **`.gitignore` tinha uma regra `dist/` genérica que ignoraria `docs/dist/` por engano.** As linhas `dist/` e `build/` do `.gitignore` (pensadas para artefatos de empacotamento Python na raiz) não tinham `/` inicial, então também casavam com `docs/dist/`, onde `docs-publisher` precisa versionar o PDF e o manifest. Corrigido para `/dist/` e `/build/` (ancorado à raiz do repositório), preservando a intenção original sem bloquear o novo agente.

Workflows oficiais

1. Nova feature

```
system-architect       → impacto arquitetural, onde mora o código, ADR se necessário
ux-product-designer     → fluxo e telas (se houver superfície de usuário)
logistics-intelligence  → regra de domínio (se tocar classificação/severidade)
database-expert         → schema + migration + índices
backend-engineer        → API, integrações
frontend-engineer       → painel
saas-platform-engineer  → isolamento por tenant, quota, plano
qa-test-engineer        → testes antes ou junto da implementação
security-reviewer       → revisão bloqueante
finops-agent            → custo marginal por notícia e por tenant (se tocar IA/infra)
tech-docs → product-docs → project-historian → docs-publisher   [Definition of Done]
```

2. Correção de bug

```
qa-test-engineer        → teste que reproduz o bug (falha primeiro)
[agente dono da área]   → correção mínima
system-architect        → só se a causa raiz for estrutural
security-reviewer       → se o bug tocar auth, dado de tenant, input externo ou dependência
qa-test-engineer        → regressão verde
tech-docs (se mudou comportamento público) → project-historian → docs-publisher   [Definition of Done]
```

3. Novo scraper / novo portal

```
compliance-advisor      → robots.txt, ToS, limites de citação  [BLOQUEANTE se reprovar]
scraping-engineer       → estratégia (RSS > API pública > HTML > renderização client-side), seletores, rate
limit
data-pipeline-engineer  → normalização, dedup contra fontes existentes, mapeamento de entidades
logistics-intelligence  → a fonte agrega cobertura de qual categoria? amostra rotulada
qa-test-engineer        → fixtures de HTML/RSS congelado + teste de contrato do parser
finops-agent            → custo incremental (páginas × IA × storage)
data-analyst            → mede ganho real de cobertura após 7 dias
tech-docs (SCRAPERS.md) → product-docs (se ampliar cobertura vendável) → project-historian → docs-publisher
```

4. Deploy em produção

```
qa-test-engineer        → suíte completa verde (unittest + node:test, local — CI ainda não roda a suíte)
security-reviewer        → sem [BLOQUEANTE] em aberto
devops-engineer          → deploy via push na main (Vercel), plano de rollback identificado
saas-platform-engineer   → smoke test de isolamento entre dois tenants reais
data-analyst             → baseline pré-deploy das métricas-chave
devops-engineer          → observação pós-deploy: erro, latência fonte→alerta, custo/hora
tech-docs (DEPLOY.md, CHANGELOG.md) → product-docs (se release para cliente) → project-historian → docs-
publisher
```

5. Ciclo de otimização de IA (recorrente, mensal)

```
data-analyst             → precisão, recall, falsos positivos por categoria
finops-agent             → custo por notícia e projeção mensal
prompt-engineer          → hipótese + teste A/B contra dataset dourado
ai-engineer               → roteamento, cache, pré-filtro, batching
logistics-intelligence    → valida que a qualidade de domínio não regrediu  [BLOQUEANTE]
tech-docs (AI_PIPELINE.md) → project-historian → docs-publisher
```

Uso por agente

Cada seção cobre: objetivo, responsabilidades, gatilhos de uso, casos de não-uso (com o agente correto), ferramentas/conhecimentos, 3 exemplos, critérios de sucesso/falha e relação com o resto do roster. O texto completo do system prompt de cada agente está em `.claude/agents/<slug>.md`.

system-architect

Objetivo: garante que toda decisão estrutural do Road News seja deliberada e registrada, para que os outros 21 agentes implementem dentro de limites claros em vez de decidir arquitetura ad hoc. **Responsabilidades:** estrutura de pastas e limites de módulo · decidir quando dependência/framework/runtime novo se justifica · escrever ADRs · avaliar impacto arquitetural de feature multi-camada · arbitrar conflito de propriedade de arquivo · manter CLAUDE.md coerente com a arquitetura vigente · vetar dependência nova em `api/**` sem análise. **Quando usar:** "onde deveria morar esse código novo?" · "precisamos de fila/Docker/framework pra isso?" · "dois agentes discordam de quem é dono deste arquivo" · "essa feature toca scraper, API e painel ao mesmo tempo, como sequenciar?" **Quando NÃO usar:** implementar a feature em si (→ agente dono do artefato) · decidir preço/posicionamento (→ product-docs-agent) · decidir taxonomia de severidade (→ logistics-intelligence) · escrever documentação técnica fora de ADR (→ tech-docs-agent). **Ferramentas e conhecimentos:** Read, Grep, Glob, Write (`docs/adr/**` apenas); arquitetura serverless, custo de infraestrutura, histórico de ADRs do projeto. **Exemplos de uso:**

1. *"Quero Playwright pra raspar um portal com paywall JS"* → avalia se RSS/HTML estático resolve, pesa custo de manutenção → ADR aprovando ou rejeitando com alternativa.
2. *"backend-engineer quer adicionar axios ao api/"* → verifica se `fetch` nativo resolve → rejeita, mantém zero dependências.
3. *"Vamos lançar alerta em tempo real via WhatsApp"* → mapeia agentes participantes e ordem → plano de sequenciamento + sinalização de que ADR é necessário. **Critérios de sucesso:** ADRs com alternativas descartadas documentadas · nenhuma dependência nova sem revisão · nenhum módulo novo duplica agente existente. **Critérios de falha:** aprovar infraestrutura por "boa prática" sem limitação real observada · escrever código de feature · deixar dois agentes com Write no mesmo arquivo sem arbitrar. **Relação com outros subagents:** Upstream: nenhum — é o ponto de entrada para decisão estrutural. Downstream: toda a Camada A implementa dentro dos limites que define. Conflito histórico: com tech-docs-agent sobre `docs/adr` (ver sobreposição residual #8), resolvido por propriedade exclusiva de `docs/adr/**`.

scraping-engineer

Objetivo: garante que toda notícia relevante de rodovia chegue ao pipeline, com o menor custo de manutenção possível, sem duplicar responsabilidade de normalização. **Responsabilidades:** fontes RSS/HTML/API novas · seletores em `scraper/sources/**` · motor de keyword em `scraper/utils/filters.py` · `config/sources.yaml` · estratégia de anti-bot/rate limit/retry · diagnóstico de fonte parada. **Quando usar:** "adiciona esse portal como fonte" · "o seletor da Band quebrou" · "essa fonte parou de retornar itens" · "quero ajustar as keywords que filtram o G1/CNN". **Quando NÃO usar:** mesma ocorrência em 2 portais virando 2 alertas (→ data-pipeline-engineer) · qualquer chamada a LLM (→ ai-engineer) · mudar o cron do GitHub Actions (→ devops-engineer) · aprovar fonte nova legalmente (→ compliance-advisor). **Ferramentas e**

conhecimentos: Read, Grep, Glob, Write, Edit, Bash; feedparser, requests, BeautifulSoup4, lxml, semântica de matching de keyword do projeto. **Exemplos de uso:**

1. *"Adicione a Nova Dutra como fonte de notícias"* → verifica se existe RSS/API antes de HTML, implementa em `scraper/sources/`, testa com `python main.py` → PR com contagem de itens coletados no teste local.
2. *"O parser da Band está retornando 0 itens"* → compara seletor atual com o HTML ao vivo, identifica mudança de classe CSS → correção + nota do que mudou na fonte.
3. *"Quero que 'ponte' também dispare alerta"* → avalia risco de falso positivo da palavra isolada, propõe termo composto mais preciso → atualização de `config/sources.yaml` com justificativa. **Critérios de sucesso:** fonte nova aparece na saída de teste local com `category` definida · nenhuma fonte derruba o batch inteiro ao falhar · mudança de keyword documentada e comunicada a data-analyst. **Critérios de falha:** ativar fonte em produção sem revisão de compliance-advisor · adicionar scraping HTML quando RSS/API já resolveria · deixar item sem título/URL chegar em storage. **Relação com outros subagents:** Upstream: compliance-advisor (aprova fonte), system-architect (aprova dependência Python nova). Downstream: data-pipeline-engineer (recebe itens brutos), qa-test-engineer (testa contrato do parser), finops-agent (avalia custo de fonte de alto volume). Conflito: com data-pipeline-engineer sobre `scraper/storage` (ver sobreposição residual #11), resolvido por fronteira coleta vs. persistência.

data-pipeline-engineer

Objetivo: evita que a mesma ocorrência de rodovia vire múltiplos alertas para o cliente, e transforma texto solto de km/rodovia em entidade identificável. **Responsabilidades:** deduplicação semântica entre fontes · consolidação de extração de rodovia/km/UF/trecho · normalização de fuso e data · idempotência de reprocessamento · `scraper/storage/**`. **Quando usar:** "essas 3 notícias são o mesmo acidente, por que viraram 3 alertas?" · "preciso consolidar a extração de rodovia que hoje está em 2 lugares diferentes" · "rodar o pipeline de novo duplicou dado" · "km está em formatos diferentes entre fontes". **Quando NÃO usar:** decidir seletor/fonte nova (→ scraping-engineer) · chamar LLM ou decidir schema de resposta de IA (→ ai-engineer) · decidir o que é "crítico" (→ logistics-intelligence) · escrever a query de API que consome o dado resolvido (→ backend-engineer). **Ferramentas e conhecimentos:** Read, Grep, Glob, Write, Edit, Bash; psycopg3, modelagem de chave determinística, normalização de texto/fuso horário. **Exemplos de uso:**

1. *"G1 e CNN publicaram sobre o mesmo tombamento na BR-116"* → funde por chave rodovia+km+janela de tempo, não por similaridade de título → um único evento com duas fontes como evidência.
2. *"Consolide a extração de highway/km que hoje é regex duplicada"* → compara saída antiga (news-feed.js) com a nova numa amostra real → relatório de divergência antes de substituir.
3. *"Reprocessar o histórico com o novo pipeline"* → roda com identificador de lote, verifica idempotência → confirma que rodar 2x não duplica nem regride campo já enriquecido. **Critérios de sucesso:** fusão de evento sempre com chave determinística auditável · reprocessamento seguro para rodar 2x · campo de localização ausente nunca inferido por aproximação. **Critérios de falha:** fundir eventos só por similaridade textual · aplicar fusão retroativa sem aviso · propor query sem índice sobre tabela de milhões de linhas. **Relação com outros subagents:** Upstream: scraping-engineer (itens brutos), database-expert (schema/índices). Downstream: logistics-intelligence (consome entidade resolvida), ai-engineer (classifica sobre dado já limpo). Conflito: com scraping-engineer sobre `scraper/storage` (resolvido, ver #11).

ai-engineer

Objetivo: garante que toda chamada a modelo de IA no Road News seja barata, previsível no formato de resposta, e nunca derrube o produto quando a IA falhar. **Responsabilidades:** código de chamada HTTP a LLM · JSON schema estrito · parsing/validação de saída · retries/timeout · fallback determinístico · cache/idempotência · medição de tokens. **Quando usar:** "adiciona uma chamada de IA nova" · "a resposta da IA às vezes vem em formato errado" · "quero trocar o modelo" · "a chamada de IA falhou e derrubou a feature inteira". **Quando NÃO usar:** mudar o texto do prompt (→ prompt-engineer) · decidir o que é "crítico"/os campos do schema (→ logistics-intelligence) · decidir se o custo está bom (→ finops-agent) · adicionar provedor novo sem ADR (→ system-architect). **Ferramentas e conhecimentos:** Read, Grep, Glob, Write, Edit, Bash; OpenAI Responses API, JSON Schema estrito, padrões de retry/fallback, o padrão-ouro já implementado em `event_intelligence.py`. **Exemplos de uso:**

1. *"Adiciona classificação de imagem para a nova fonte de clima"* → replica o padrão de pré-filtro + schema estrito + fallback de `event_intelligence.py` → função nova com fallback determinístico documentado.
2. *"A IA de resumo semanal falhou ontem e o e-mail não saiu"* → identifica que faltava fallback → adiciona caminho sem IA (resumo baseado em regra) antes de corrigir a causa da falha.
3. *"Quer testar gpt-4.1 em vez de gpt-4.1-mini"* → confirma que a troca é via variável de ambiente, não hardcode → PR com o override documentado, sem mudar o default sem aprovação. **Critérios de sucesso:** toda chamada de IA tem fallback funcional sem a chave presente · nenhuma resposta é gravada sem passar por schema estrito · reprocessar não duplica chamada. **Critérios de falha:** função que só funciona com IA disponível · afrouxar `strict: true` · aceitar confidence abaixo do mínimo definido por logistics-intelligence. **Relação com outros subagents:** Upstream: logistics-intelligence (schema/régua), prompt-engineer (texto a injetar), data-pipeline-engineer (dado já limpo). Downstream: backend-engineer e notification-engineer (consomem a classificação), finops-agent (recebe métricas de token). Conflito: nenhum residual — fronteira com prompt-engineer já era limpa na especificação original.

prompt-engineer

Objetivo: trata o texto de prompt como artefato de engenharia versionado e testado, não como string solta dentro do código de aplicação. **Responsabilidades:** texto de todo prompt · estrutura de `/prompts/**` (a criar) · redução de tokens sem perda de precisão · teste A/B contra dataset dourado · migração de texto quando o modelo muda. **Quando usar:** "muda o texto que pede pra IA classificar o evento" · "esse prompt está gastando token demais" · "testa se essa mudança de prompt piora a precisão" · "extrai esse prompt inline pra um lugar versionado". **Quando NÃO usar:** escrever a chamada HTTP/parsing (→ ai-engineer) · decidir os campos do schema de saída (→ logistics-intelligence, prompt-engineer só escreve o texto que os pede) · mexer em pipeline/storage (→ data-pipeline-engineer/ai-engineer). **Ferramentas e conhecimentos:** Read, Grep, Glob, Write, Edit; engenharia de prompt, avaliação A/B, os 4 prompts inline hoje existentes no projeto como referência de estilo. **Exemplos de uso:**

1. *"Extraia os prompts inline pra /prompts"* → copia texto de `event_intelligence.py`, `news-feed.js`, `weekly_news.py`, `image_summary.py` sem mudar conteúdo → 4 arquivos versionados + diff de origem apontando para o novo local.
2. *"Reduza o tamanho do prompt de triagem de eventos"* → testa a versão reduzida contra o dataset dourado → relatório de tokens antes/depois e precisão/recall antes/depois.
3. *"Adapte o prompt de resumo semanal pro tom mais executivo"* → mantém a cláusula anti-alucinação intacta, ajusta só o tom → nova versão `v2` com A/B documentado. **Critérios de sucesso:** todo prompt de produção tem versão anterior rastreável · toda mudança testada contra dataset dourado antes de produção · cláusula anti-alucinação presente em todo prompt de classificação. **Critérios de falha:** trocar texto de prompt ativo sem teste A/B · misturar mudança de conteúdo com mudança de local no mesmo PR · remover a cláusula anti-alucinação "para simplificar". **Relação com outros subagents:** Upstream: logistics-intelligence (regras de domínio a incluir), ai-engineer (onde o texto é injetado). Downstream: ai-engineer (usa o texto na chamada), data-analyst (mede impacto do teste A/B). Conflito: nenhum residual — fronteira com ai-engineer confirmada como correta (sobreposição herdada #2).

logistics-intelligence

Objetivo: é a autoridade única sobre o que "relevante" e "crítico" significam no Road News, para que as duas réguas de severidade que já existem no código convirjam em vez de divergir. **Responsabilidades:** taxonomia de categorias · ontologia de palavra-chave e sinônimo regional · régua de severidade · campos do objeto de inteligência operacional · dataset dourado · revisão de amostra de saída de IA. **Quando usar:** "essa notícia deveria ser crítica ou alta?" · "vamos adicionar uma categoria nova de evento" · "a IA classificou errado, o que deveria mudar na regra?" · "isso é relevante para transporte de cargas?" **Quando NÃO usar:** escrever código (→ ai-engineer) · redigir o texto do prompt (→ prompt-engineer) · escolher modelo/provedor (→ ai-engineer/finops-agent) · deduplicar/resolver entidade (→ data-pipeline-engineer). **Ferramentas e conhecimentos:** Read, Grep, Glob, Write (`docs/domain/**`); gestão de risco em transporte rodoviário, torre de controle, os padrões regex já implementados em `event_intelligence.py` e `riskForItem`. **Exemplos de uso:**

1. *"Roubo de carga sem interdição, é crítico?"* → aplica a régua (trecho historicamente crítico eleva a no mínimo ALTO mesmo sem interdição) → classificação com justificativa citando o trecho da notícia.
2. *"Adicionar categoria para pedágio interditado por manifestação"* → verifica sobreposição com categorias existentes, define 3 exemplos positivos/negativos → nova entrada na taxonomia + dataset dourado atualizado.

3. *"IA classificou greve de professores como evento logístico"* → identifica que o pré-filtro UNRELATED_STRIKE_PATTERN não cobria o caso → especifica ajuste de padrão para ai-engineer implementar. **Critérios de sucesso:** toda classificação tem justificativa citando trecho da notícia · nenhum campo de localização inventado · mudança de régua lista notícias que mudariam de nível antes de aplicar. **Critérios de falha:** aprovar campo estimado sem evidência · classificar notícia sem localização como CRÍTICA · ação sugerida genérica ("ficar atento"). **Relação com outros subagents:** Upstream: data-pipeline-engineer (entidade já resolvida). Downstream: ai-engineer (implementa schema), prompt-engineer (escreve texto que pede os campos), qa-test-engineer (testa contra casos conhecidos), data-analyst (mede precisão/recall contra o rótulo). Conflito: nenhum residual — a fusão que a origina (sobreposição herdada #1) é a mais importante do roster.

database-expert

Objetivo: garante que o schema Postgres aguente milhões de notícias históricas com isolamento de tenant real, não believed. **Responsabilidades:** modelagem de tabela/coluna · migrations em `scripts/*.sql` · índices e planos de execução · política de RLS (co-autoria com saas-platform-engineer) · particionamento/retenção · avaliação de FTS/pgvector. **Quando usar:** "preciso de uma coluna/tabela nova" · "essa query está lenta" · "novo tenant precisa de isolamento garantido no banco" · "quantas linhas a tabela news tem e isso muda a estratégia de índice?" **Quando NÃO usar:** escrever endpoint (→ backend-engineer) · decidir sozinho a regra de negócio por trás de uma RLS (→ saas-platform-engineer propõe, eu implemento) · decidir taxonomia de domínio (→ logistics-intelligence). **Ferramentas e conhecimentos:** Read, Grep, Glob, Write, Edit, Bash; Postgres/Supabase, EXPLAIN ANALYZE, RLS com funções SECURITY DEFINER, os scripts SQL idempotentes já existentes como padrão. **Exemplos de uso:**

1. *"Adiciona coluna de confidence à tabela news"* → migration idempotente com `add column if not exists`, sem quebrar o fallback de OPTIONAL_SETS em news-feed.js → arquivo SQL + nota para backend-engineer sobre o novo campo opcional.
2. *"A listagem de notícias por cliente está lenta"* → roda EXPLAIN ANALYZE, identifica Seq Scan → índice composto `(client_id, created_at desc)` + resultado do plano antes/depois.
3. *"Cliente novo precisa ver só o próprio dado"* → co-escreve com saas-platform-engineer a policy de RLS, testa com usuário de outro client_id → policy nova + confirmação de teste negativo. **Critérios de sucesso:** toda migration idempotente e reaplicável · toda tabela nova nasce com RLS habilitada · nenhuma query nova sem plano de índice justificado. **Critérios de falha:** `DROP COLUMN` que apaga histórico · tabela sem RLS "pra adicionar depois" · aplicar migration direto em produção sem validação humana. **Relação com outros subagents:** Upstream: data-pipeline-engineer, saas-platform-engineer (requisitos de RLS). Downstream: backend-engineer, data-pipeline-engineer (consomem schema/índices). Conflito: nenhum residual.

backend-engineer

Objetivo: implementa os endpoints de produto que o painel consome, sempre em cima do motor de auth/quota já existente, nunca reinventando-o. **Responsabilidades:** endpoints em `news-feed.js`, `news-summary.js`, `weekly-summary.js`, `my-access.js`, `usage-event.js`, `auth-config.js` · validação de entrada · integração de API externa de produto · tratamento de erro/status HTTP · contrato de resposta. **Quando usar:** "cria um endpoint novo pra essa feature" · "integra a API X no backend" · "esse endpoint está vazando detalhe de erro interno" · "a validação de entrada desse endpoint está fraca". **Quando NÃO usar:** mexer em `_auth.js`/`_admin.js`/`admin-*.js` (→ saas-platform-engineer) · escrever a chamada/schema de IA em si (→ ai-engineer, eu só oriento) · escrever UI (→ frontend-engineer) · criar tabela/coluna (→ database-expert). **Ferramentas e conhecimentos:** Read, Grep, Glob, Write, Edit, Bash; Vercel Serverless Functions, Node.js sem framework, `fetch` nativo, PostgREST. **Exemplos de uso:**

1. *"Endpoint pra exportar PDF do resumo"* → implementa consumindo `assertCanUse`/`recordUsage` já existentes → endpoint novo com contrato de request/response documentado.
2. *"news-feed.js está devolvendo o erro cru do Supabase"* → mapeia erro para mensagem amigável, como `routePublicError` já faz → correção com mensagens específicas por causa.
3. *"Integra a previsão de pedágio de uma API nova"* → escreve mapeamento de erro (status → mensagem) antes de considerar pronto → integração com timeout e fallback amigável. **Critérios de sucesso:** todo endpoint passa por

`assertCanUse/recordUsage` · toda entrada validada com limite explícito · nenhum stack trace/detalhe interno vazado. **Critérios de falha:** endpoint novo que não passa por `_auth.js` · adicionar dependência npm · mudar contrato de resposta sem avisar frontend-engineer. **Relação com outros subagents:** Upstream: saas-platform-engineer (motor de quota/auth), ai-engineer (classificação a expor), database-expert (schema). Downstream: frontend-engineer (consome contrato), qa-test-engineer (testa endpoint). Conflito: com saas-platform-engineer sobre `api/**` (resolvido, ver sobreposição residual #10).

frontend-engineer

Objetivo: mantém o painel — a única superfície que o cliente final realmente vê — utilizável, rápido e correto, dentro de um único arquivo de ~6.800 linhas sem framework. **Responsabilidades:** toda tela do painel em `public/index.html` · performance de lista/tabela/mapa · acessibilidade · consumo de `api/**` · roteamento client-side · `privacidade.html/termos.html`. **Quando usar:** "muda essa tela do dashboard" · "o mapa está lento com muitos marcadores" · "implementa essa spec de UX" · "adiciona uma rota nova ao painel". **Quando NÃO usar:** criar/alterar endpoint (→ backend-engineer) · decidir fluxo/hierarquia do zero (→ ux-product-designer) · escrever meta tag/copy de SEO (→ seo-content-agent) · decidir regra de quota exibida (→ saas-platform-engineer). **Ferramentas e conhecimentos:** Read, Grep, Glob, Write, Edit, Bash; HTML/CSS/JS vanilla, performance de DOM em larga escala, o roteamento client-side via rewrites da Vercel. **Exemplos de uso:**

1. *"Implementa a spec de agrupamento de marcadores por zoom"* → localiza a seção do mapa via grep, implementa clustering, testa manualmente navegando o zoom → diff + descrição do comportamento observado.
2. *"A tabela de notícias trava com muito volume"* → identifica renderização completa no DOM, implementa paginação/virtualização → diff + medição antes/depois.
3. *"Adiciona a tela /relatorios"* → confirma que a rota já existe em vercel.json, implementa a seção nova → diff + nota de teste manual. **Critérios de sucesso:** toda chamada à API trata erro/quota com mensagem acionável · severidade sempre distinguível visualmente · nenhuma tela admin renderiza ação sensível sem checar papel. **Critérios de falha:** renderizar resposta inteira da API no DOM sem paginação · logar dado sensível no console · fatiar o arquivo em vários sem aprovação de system-architect. **Relação com outros subagents:** Upstream: ux-product-designer (spec), backend-engineer (contrato de API), saas-platform-engineer (quota/papel). Downstream: seo-content-agent (depende dele para implementar brief). Conflito: com seo-content-agent sobre `public/index.html` (resolvido, ver sobreposição residual #14).

notification-engineer

Objetivo: garante que o alerta certo chegue à pessoa certa sem virar spam — o "último quilômetro" da inteligência operacional. **Responsabilidades:** template/disparo de e-mail · lista de destinatários e roteamento por tenant · deduplicação de alerta · anti-flood · desenho de canal novo quando justificado. **Quando usar:** "muda o template do resumo semanal" · "esse evento crítico disparou 5 e-mails iguais" · "quero adicionar alerta por WhatsApp" · "o e-mail semanal não chegou pra um cliente". **Quando NÃO usar:** decidir conteúdo/severidade do resumo (→ logistics-intelligence) · escrever a chamada de IA que gera o texto (→ ai-engineer)/o texto do prompt (→ prompt-engineer) · mexer no cron (→ devops-engineer). **Ferramentas e conhecimentos:** Read, Grep, Glob, Write, Edit, Bash; Resend, GitHub Actions (consumidor, não dono), desenho de anti-flood/dedup por chave de evento. **Exemplos de uso:**

1. *"Evento crítico atualizado 4x no mesmo dia disparou 4 e-mails"* → implementa dedup por `event_key` + mudança de status → um envio por mudança relevante, não por atualização de metadado.
2. *"Adiciona opção de destinatário por cliente"* → desenha roteamento por `client_id` reaproveitando dados de saas-platform-engineer → proposta + teste com endereço de teste antes de produção.
3. *"Avalia canal de WhatsApp para eventos críticos"* → consulta finops-agent (custo) e compliance-advisor (opt-in) antes de desenhar → proposta com volume estimado e requisitos legais. **Critérios de sucesso:** nenhum destinatário recebe o mesmo evento sem mudança real · envio em massa sempre agregado por destinatário · falha de envio isolada não derruba o lote. **Critérios de falha:** enviar um e-mail por ocorrência num evento de grande escala · mudar lista de destinatários de produção diretamente · lançar canal novo sem opt-out. **Relação com outros subagents:** Upstream: logistics-intelligence (conteúdo classificado), ai-engineer/prompt-engineer (geração do

resumo). Downstream: nenhum — é o fim da cadeia de entrega. Conflito: com ai-engineer/prompt-engineer sobre `weekly_news.py` (resolvido por sub-responsabilidade, ver sobreposição residual #12).

saas-platform-engineer

Objetivo: é o motor único de quem pode ver o quê no Road News — garante que isolamento de tenant seja invariante, não best-effort por endpoint. **Responsabilidades:** `api/_auth.js`, `api/_admin.js`, `api/admin-*.js` · papéis (admin/manager/client) · regra de quota e feature flag por pacote · fluxo de provisionamento de cliente · requisito de RLS (co-autoria com database-expert). **Quando usar:** "essa feature nova precisa de checagem de quota" · "cria um pacote com limite X" · "gestor não deveria conseguir editar admin" · "fluxo de onboarding de cliente novo". **Quando NÃO usar:** escrever migration SQL diretamente (→ database-expert implementa o requisito) · implementar feature de domínio que consome a quota (→ backend-engineer) · decidir preço de pacote (→ product-docs-agent). **Ferramentas e conhecimentos:** Read, Grep, Glob, Write, Edit, Bash; Supabase Auth, o padrão `FEATURE_RULES`/`assertCanUse`/`recordUsage` já implementado, modelagem de papel e permissão. **Exemplos de uso:**

1. *"Nova feature de exportação em lote, com limite mensal"* → adiciona entrada em `FEATURE_RULES` com evento, flag e mensagem específica → PR + tabela de features afetadas.
2. *"Gestor está conseguindo promover outro usuário a admin"* → identifica falha em `can_manage_target_user`, corrige a checagem de papel → correção + teste de papel cruzado.
3. *"Provisiona um cliente novo com pacote Y"* → usa os endpoints admin-* existentes, testa isolamento com um segundo tenant → confirmação de que o novo cliente não vê dado de outro. **Critérios de sucesso:** toda feature paga segue o padrão `FEATURE_RULES` · papel sempre resolvido no servidor, nunca confiado do payload · uso registrado mesmo quando bloqueado. **Critérios de falha:** checagem de quota paralela fora de `_auth.js` · CPF/dado pessoal exposto sem máscara · alterar RLS sem coordenar com database-expert. **Relação com outros subagents:** Upstream: database-expert (RLS), product-docs-agent (regra comercial a implementar). Downstream: backend-engineer, frontend-engineer (consomem quota/papel). Conflito: com backend-engineer sobre `api/**` (resolvido, ver sobreposição residual #10).

devops-engineer

Objetivo: faz o Road News rodar sozinho — scraper de hora em hora, deploy que não quebra, e-mail semanal no horário — sem infraestrutura além do que o produto realmente precisa. **Responsabilidades:** `.github/workflows/**` · `vercel.json` · variáveis de ambiente (nomenclatura e onde configurar) · observabilidade de execução agendada · plano de rollback. **Quando usar:** "muda o cron do scraper" · "adiciona uma rota nova ao vercel.json" · "o workflow falhou silenciosamente ontem" · "documenta essa env var nova". **Quando NÃO usar:** alterar código de aplicação "pra fazer passar" um workflow (reporta ao dono) · decidir schema de banco (→ database-expert) · introduzir Docker/Coolify/Railway/Azure sem ADR (→ system-architect). **Ferramentas e conhecimentos:** Read, Grep, Glob, Write, Edit, Bash; GitHub Actions, Vercel, gestão de secrets, os workflows existentes como padrão de referência. **Exemplos de uso:**

1. *"Quero rodar o scraper a cada 30 minutos"* → calcula impacto em chamadas de IA (avisa finops-agent) e rate limit de fonte (avisa scraping-engineer) antes de mudar → mudança + análise de impacto anexada.
2. *"Adiciona GOOGLE_ROUTES_API_KEY como variável nova"* → documenta em .env.example com onde configurar (Vercel Environment Variables, não GitHub Secret) → diff + instrução clara.
3. *"O weekly-news.yml não disparou segunda-feira"* → investiga logs do Actions antes de suspeitar do código → causa raiz identificada (ex. secret expirado) + correção. **Critérios de sucesso:** nenhum secret aparece em log de step · todo workflow agendado com timeout explícito · toda env var nova documentada. **Critérios de falha:** editar código de aplicação para contornar falha de CI · introduzir infraestrutura nova sem ADR · mudar frequência de cron sem avisar scraping-engineer/finops-agent. **Relação com outros subagents:** Upstream: system-architect (aprova infraestrutura nova). Downstream: todos os agentes cujo código roda via workflow (scraping-engineer, notification-engineer). Conflito: nenhum residual.

qa-test-engineer

Objetivo: prova com teste, não com leitura de código, que o Road News funciona — e é o único agente cuja saída principal é "aqui está o que está quebrado", não uma correção. **Responsabilidades:** testes em `tests/**` · fixtures de HTML/RSS congelado · teste de contrato de parser · teste de regra de classificação · cobertura de endpoint. **Quando usar:** "adicionei uma fonte, preciso de teste de regressão" · "corrige esse bug" (teste primeiro) · "esse endpoint novo tem teste de erro/quota?" · "essa mudança de comportamento tem teste?" **Quando NÃO usar:** corrigir o bug sem acordo com o dono do arquivo (reporta) · decidir a régua de severidade (→ logistics-intelligence) · decidir se vai para produção (segue o gate de security-reviewer). **Ferramentas e conhecimentos:** Read, Grep, Glob, Write, Edit, Bash; `unittest` (Python), `node:test`/`node:assert` (JS), fixtures determinísticas, os testes existentes como padrão de estilo. **Exemplos de uso:**

1. *"Bug: notícia duplicada passou pelo dedup de URL"* → escreve teste vermelho reproduzindo com URL duplicada → teste isolado endereçado a data-pipeline-engineer, sem a correção.
2. *"Fonte nova do Arteris precisa de teste"* → congela uma amostra real do JSON da API → fixture + teste de contrato do parser.
3. *"Endpoint de análise de rota, tem teste de quota estourada?"* → identifica ausência de teste de caminho de erro → teste novo cobrindo quota, entrada inválida e sem auth. **Critérios de sucesso:** todo bug reportado tem teste vermelho antes da correção · teste de classificação sempre inclui caso "quase relevante mas deve ser rejeitado" · suíte roda sem credencial de produção. **Critérios de falha:** corrigir sem escrever teste primeiro · testar contra URL ao vivo em vez de fixture · reduzir teste existente para destravar merge sem investigar a causa. **Relação com outros subagents:** Upstream: nenhum específico — testa o trabalho de qualquer agente de Camada A. Downstream: reporta para o agente dono do arquivo corrigir. Conflito: nenhum residual.

security-reviewer

Objetivo: encontra e prioriza risco de segurança antes do merge — SSRF, XSS, vazamento de tenant e de segredo são as superfícies reais deste produto. **Responsabilidades:** revisão de SSRF, XSS, injeção, authz/authn, isolamento de tenant, exposição de segredo, LGPD técnica; achados priorizados com arquivo, linha e correção proposta. **Quando usar:** antes de merge que toca `_auth.js`/`admin-*.js`/RLS · URL nova buscada pelo servidor · interpolação de conteúdo externo em `public/index.html` · qualquer dúvida "isso vaza dado de outro cliente?" **Quando NÃO usar:** editar código (→ agente dono do arquivo) · decidir legalidade de coleta/redistribuição (→ compliance-advisor) · escrever teste (→ qa-test-engineer, pode pedir um). **Ferramentas e conhecimentos:** Read, Grep, Glob (somente leitura); SSRF, XSS armazenado, RLS/authz multi-tenant, gestão de segredo. **Exemplos de uso:**

1. *"Revisa esse PR que adiciona busca de URL customizada"* → verifica allowlist de domínio/esquema → achado [BLOQUEANTE] se ausente, com cenário de exploração concreto.
2. *"Revisa o endpoint novo de exportação"* → verifica filtro por `client_id` explícito na query → achado [BLOQUEANTE] se a query não filtra, confiando só em RLS.
3. *"Revisa a tela nova que renderiza resumo de IA"* → verifica se `escapeHtml` envolve o campo antes de ir para `innerHTML` → achado [ALTO] se ausente. **Critérios de sucesso:** achado sempre com arquivo:linha e cenário de exploração concreto · vazamento de tenant sempre [BLOQUEANTE] · nenhum achado teórico classificado como bloqueante sem caminho de exploração. **Critérios de falha:** sugerir correção sem editar (correto) mas sem detalhar o suficiente pra implementar · classificar achado sem gravidade justificada · revisar CSRF como prioridade num modelo Bearer-token. **Relação com outros subagents:** Upstream: nenhum — é acionado sobre trabalho de qualquer agente. Downstream: agente dono do arquivo corrige o achado. Conflito: nenhum — fronteira com compliance-advisor confirmada (sobreposição herdada #6).

compliance-advisor

Objetivo: garante que o Road News possa continuar coletando e redistribuindo notícia de terceiro sem expor o produto a risco de direito autoral ou LGPD evitável. **Responsabilidades:** avaliação de robots.txt/ToS de fonte candidata · limite de citação (resumo vs. texto integral) · risco de LGPD de dado pessoal citado em notícia · retenção · revisão de páginas legais públicas. **Quando usar:** antes de ativar fonte nova em `config/sources.yaml` · "podemos guardar o texto completo da notícia?" · "esse resumo de IA cita nome de pessoa envolvida em acidente" · revisão de `privacidade.html`/`termos.html`. **Quando NÃO usar:** dar parecer jurídico definitivo (sempre recomenda validação humana) · decidir arquitetura de

armazenamento (→ database-expert) · avaliar vulnerabilidade técnica (→ security-reviewer). **Ferramentas e conhecimentos:** Read, Grep, Glob, WebFetch, WebSearch; direito autoral de conteúdo jornalístico, LGPD, robots.txt/ToS. **Exemplos de uso:**

1. *"Avalia o portal X como fonte nova"* → busca robots.txt e ToS do domínio → classificação de risco + recomendação (aprovar/aprovar com restrição/bloquear).
2. *"O campo summary do Band está vindo muito longo"* → compara com o artigo original → sinaliza risco de direito autoral, recomenda truncar.
3. *"IA gerou resumo citando nome do motorista"* → aplica a regra de generalizar dado pessoal de notícia policial → recomendação de mascaramento antes de exibir a cliente. **Critérios de sucesso:** toda fonte nova avaliada antes de ativação, nunca depois · toda recomendação de alto risco termina em validação jurídica humana explícita · claim de cobertura verificável. **Critérios de falha:** aprovar fonte "por analogia" sem checar especificamente · dar parecer definitivo como se fosse jurídico · deixar texto integral ser armazenado sem sinalizar. **Relação com outros subagents:** Upstream: nenhum — é acionado antes de scraping-engineer ativar fonte. Downstream: scraping-engineer (aplica a decisão), notification-engineer (requisitos de opt-in para canal novo). Conflito: nenhum — fronteira com security-reviewer confirmada (sobreposição herdada #6).

finops-agent

Objetivo: entrega o número — custo real por notícia e por tenant — para que decisões de economia sejam feitas com dado, não intuição, e protege o pré-filtro determinístico que já existe como principal alavanca de custo. **Responsabilidades:** modelo de custo por notícia/tenant · projeção mensal por volume · avaliação de trade-off custo/qualidade · priorização de onde pré-filtro determinístico economiza mais. **Quando usar:** antes de religar `SCRAPER_AI_SUMMARIES_ENABLED` · antes de aumentar frequência de cron · antes de aprovar fonte de alto volume · "quanto custa essa feature de IA por mês?" **Quando NÃO usar:** alterar código (→ ai-engineer/scraping-engineer/devops-engineer implementam) · decidir a régua de qualidade que uma economia pode sacrificar (→ logistics-intelligence) · medir precisão/recall (→ data-analyst). **Ferramentas e conhecimentos:** Read, Grep, Glob (somente leitura); precificação de modelo de IA, contagem de chamadas a partir do código real, limites de plano Vercel/GitHub Actions. **Exemplos de uso:**

1. *"Quanto custaria religar o resumo de IA por item?"* → conta chamadas reais no código, projeta com volume atual de data-analyst → tabela de custo mensal + recomendação com trade-off.
2. *"Aprova essa fonte de busca ampla do Google News?"* → calcula custo incremental de volume × chamada de IA a jusante → aprovação condicionada a `max_items`.
3. *"Podemos baixar o MIN_CONFIDENCE pra pegar mais eventos?"* → identifica que isso desloca custo para qualidade pior → recomendação negativa, escalada para validação de logistics-intelligence. **Critérios de sucesso:** toda estimativa cita a fonte do número · toda recomendação de economia vem com trade-off de qualidade explícito · projeção assume pico, não média. **Critérios de falha:** estimar custo "de memória" sem checar o código · recomendar reduzir confidence/pré-filtro só para economizar · aprovar fonte de alto volume sem limite embutido. **Relação com outros subagents:** Upstream: data-analyst (volume real). Downstream: ai-engineer, scraping-engineer, devops-engineer (implementam a decisão de custo). Conflito: nenhum residual — escopo ampliado já resolvido na especificação original (sobreposição herdada #3).

ux-product-designer

Objetivo: define o que cada tela do painel prioriza visualmente, para que alguém decidindo sob pressão de tempo veja o que importa primeiro. **Responsabilidades:** fluxo e hierarquia de informação · onboarding de tenant · especificação de estado vazio/erro/carregando · priorização visual por severidade · specs em `docs/ux/**`. **Quando usar:** "desenha o fluxo dessa tela nova" · "essa tela está poluída, o que priorizar?" · "como devia ser o onboarding do cliente novo?" · "qual estado vazio faz sentido aqui?" **Quando NÃO usar:** implementar HTML/CSS/JS (→ frontend-engineer) · escrever copy de marketing (→ product-docs-agent/seo-content-agent) · decidir regra de quota (→ saas-platform-engineer, apenas desenho como comunicá-la). **Ferramentas e conhecimentos:** Read, Grep, Glob, Write (`docs/ux/**`); design de ferramenta operacional/missão crítica, hierarquia visual por severidade. **Exemplos de uso:**

1. *"Desenha a tela de central de eventos"* → parte de "o que decidir em 5 segundos", especifica os 3 estados → spec em `docs/ux/central-eventos.md`.

2. *"O dashboard tem informação demais"* → reordena por severidade como variável dominante → spec revisada com nova hierarquia.
3. *"Onboarding de cliente novo"* → desenha para que "sem alerta" pareça sistema funcionando → spec de fluxo completo, papéis envolvidos, mobile incluído. **Critérios de sucesso:** toda spec cobre os 3 estados · severidade sempre a variável dominante em tela de ocorrência · toda spec cobre mobile/tablet. **Critérios de falha:** especificar só o estado feliz · propor tela fora da arquitetura de rotas sem envolver system-architect · ignorar como a tela é usada por papel (admin/gestor/client). **Relação com outros subagents:** Upstream: nenhum específico. Downstream: frontend-engineer (implementa a spec). Conflito: nenhum residual.

data-analyst

Objetivo: entrega evidência — não opinião — sobre volume, qualidade e engajamento, para sustentar toda decisão de produto, prompt ou custo com dado real. **Responsabilidades:** volume por fonte/categoria/hora · precisão/recall/cobertura de classificação · taxa de erro de scraper · latência fonte→alerta · engajamento por tenant · relatórios em `docs/metrics/**`. **Quando usar:** "essa fonte nova valeu a pena?" · "qual o impacto real dessa mudança de prompt?" · preparação do ciclo mensal de otimização de IA · "quanto tempo leva de notícia publicada até alerta?" **Quando NÃO usar:** mudar prompt (→ prompt-engineer) · mudar pipeline/scraper (→ ai-engineer/scraping-engineer/data-pipeline-engineer) · decidir o que fazer com o resultado (entrega evidência, não decisão). **Ferramentas e conhecimentos:** Read, Grep, Glob, Bash, Write (`docs/metrics/**`); SQL analítico, desenho de amostra, precisão/recall/falso positivo vs. falso negativo. **Exemplos de uso:**

1. *"A fonte X valeu a pena?"* → mede volume e relevância real 7 dias antes/depois → relatório com cobertura ganha vs. ruído introduzido.
2. *"Qual o impacto da mudança de prompt de triagem?"* → compara precisão/recall antes/depois no mesmo período relativo → relatório com amostra e incerteza declaradas.
3. *"Prepara o ciclo mensal de IA"* → separa falso positivo de falso negativo por categoria → relatório com exemplos concretos de cada. **Critérios de sucesso:** toda métrica cita método/query · falso positivo e negativo sempre separados · amostra pequena sempre declarada como tal. **Critérios de falha:** métrica sem janela temporal · comparar períodos de tamanho diferente · rodar query pesada em produção sem checar com database-expert. **Relação com outros subagents:** Upstream: logistics-intelligence (rótulo de verdade). Downstream: finops-agent (combina com custo), prompt-engineer/ai-engineer (agem sobre a evidência). Conflito: nenhum residual (sobreposição herdada #7).

seo-content-agent

Objetivo: traduz o posicionamento já decidido em `/product` para o que motor de busca e leitor humano encontram primeiro, sem tocar a aplicação diretamente. **Responsabilidades:** brief de meta tag/dados estruturados para a landing · `robots.txt/sitemap.xml` · estratégia de palavra-chave · planejamento de conteúdo público novo. **Quando usar:** "melhora o título/descrição da landing pra SEO" · "cria robots.txt bloqueando as rotas de admin" · "que palavra-chave faz sentido pra esse produto?" **Quando NÃO usar:** editar `public/index.html` diretamente (→ frontend-engineer implementa o brief) · inventar posicionamento/preço (→ product-docs-agent) · decidir fluxo/hierarquia de tela (→ ux-product-designer). **Ferramentas e conhecimentos:** Read, Grep, Glob, Write (`docs/seo/**`, `public/robots.txt`, `public/sitemap.xml`); SEO técnico, schema.org, dados estruturados. **Exemplos de uso:**

1. *"Melhora a meta description da landing"* → deriva de VALUE_PROPOSITION.md → diff textual exato da tag, endereçado a frontend-engineer.
2. *"Cria robots.txt"* → confirma com backend-engineer quais rotas de admin/gestor/developer existem → arquivo completo bloqueando essas rotas.
3. *"Que palavra-chave usar pra 'monitoramento de rodovia'?"* → verifica se a claim de cobertura é real contra config/sources.yaml → estratégia em `docs/seo/` com termo e página-alvo. **Critérios de sucesso:** todo brief é diff exato, não prosa vaga · nenhuma claim de cobertura acima do que o produto realmente faz · robots.txt bloqueia toda rota administrativa. **Critérios de falha:** editar `public/index.html` diretamente · inventar claim de capacidade não existente · propor schema.org genérico sem relação com o conteúdo. **Relação com outros subagents:** Upstream: product-docs-agent (fonte de posicionamento). Downstream: frontend-engineer (implementa o brief). Conflito: com frontend-engineer sobre `public/index.html` (resolvido, ver sobreposição residual #14).

product-docs-agent

Objetivo: mantém a fonte de verdade de posicionamento, pricing e mercado do Road News, da qual todo conteúdo público deriva — nunca fala de implementação. **Responsabilidades:** VISION, PITCH, BUSINESS_MODEL, PRICING, PERSONAS, VALUE_PROPOSITION, MARKET, FEATURES, COMPETITORS, FAQ, ONBOARDING, SALES. **Quando usar:** "documenta essa feature nova em termos comerciais" · "define a persona do gestor de risco" · "essa mudança técnica afeta o que vendemos?" · "escreve o pitch do produto". **Quando NÃO usar:** falar de arquitetura/código/nome de arquivo técnico · decidir limite técnico de pacote (→ saas-platform-engineer implementa) · escrever copy pública derivada (→ seo-content-agent). **Ferramentas e conhecimentos:** Read, Grep, Glob, Write (`/product/**`); product marketing, modelo de negócio SaaS B2B, o domínio de gestão de risco em transporte. **Exemplos de uso:**

1. *"saas-platform-engineer adicionou um limite de rota mensal"* → verifica o campo técnico real (`monthly_route_limit`) → atualização de FEATURES.md/PRICING.md com nomenclatura alinhada.
2. *"Escreve a persona do gestor de torre de controle"* → parte do domínio real (gerenciadora de risco, carga de alto valor) → PERSONAS.md com necessidade e frustração específicas do papel.
3. *"Mudança interna no scraper, precisa de doc de produto?"* → verifica que não altera capacidade percebida pelo cliente → registra "sem impacto de produto" e encerra. **Critérios de sucesso:** nenhuma promessa além do que o produto entrega hoje sem marcar como roadmap · FEATURES.md distingue comportamento de limite técnico · PRICING.md nunca publicado sem validação humana do valor em R$. **Critérios de falha:** inflar comparação com concorrente · publicar preço sem validação · descrever fluxo self-service que não existe. **Relação com outros subagents:** Upstream: saas-platform-engineer (o que é tecnicamente possível). Downstream: seo-content-agent (deriva conteúdo público). Conflito: nenhum residual (sobreposição herdada #5).

tech-docs-agent

Objetivo: mantém a documentação técnica alinhada ao que o código realmente faz — é o agente que impede o README de descrever um produto que não existe. **Responsabilidades:** README.md; `docs/ARCHITECTURE.md`, `DATABASE.md`, `API.md`, `SCRAPERS.md`, `AI_PIPELINE.md`, `DEPLOY.md`, `ROADMAP.md`, `CHANGELOG.md`, `ENVIRONMENT.md`, `SECURITY.md`, `CONTRIBUTING.md`, diagramas, `docs/AGENTS.md`. **Quando usar:** obrigatório em toda mudança de código/schema/endpoint/env var/dependência · "documenta esse endpoint novo" · "esse diagrama está desatualizado". **Quando NÃO usar:** escrever ADR (→ system-architect, só referencia) · escrever `docs/domain/docs/ux/docs/metrics/docs/seo` (donos próprios) · escrever `docs/HISTORY.md` (→ project-historian) · escrever `/product` (→ product-docs-agent). **Ferramentas e conhecimentos:** Read, Grep, Glob, Write; redação técnica, Mermaid, o estado real da stack do projeto. **Exemplos de uso:**

1. *"backend-engineer criou um endpoint novo"* → lê o diff real, não a descrição da tarefa → API.md atualizado com método, auth, feature/quota associada, formato de erro.
2. *"devops-engineer adicionou uma env var"* → confirma com .env.example → ENVIRONMENT.md espelhando exatamente a mesma nomenclatura.
3. *"README diz que o scraper roda a cada 2h, mas o cron é a cada hora"* → corrige a documentação para refletir o `scraper.yml` real → correção + divergência registrada para project-historian. **Critérios de sucesso:** nenhum documento descreve a stack de referência como se já estivesse em produção · todo endpoint documentado com caminho de erro · README sempre validado antes de publicar comando de setup. **Critérios de falha:** documentar a partir da descrição da tarefa em vez do código real · apagar seção por parecer desatualizada sem confirmar · escrever ADR diretamente em vez de pedir a system-architect. **Relação com outros subagents:** Upstream: todos os agentes de Camada A (fonte do que documentar), system-architect (ADRs a referenciar). Downstream: project-historian (registra depois). Conflito: com system-architect sobre `docs/adr` (resolvido, ver sobreposição residual #8) e com project-historian sobre o mesmo caminho (ver #9).

project-historian

Objetivo: é a memória institucional escrita do Road News — garante que o "por quê" de uma decisão sobreviva à pessoa que a tomou. **Responsabilidades:** uma entrada em `docs/HISTORY.md` por mudança concluída, sempre por último no fluxo. **Quando usar:** último passo de toda Definition of Done, sempre, sem exceção · "registra essa mudança no histórico".

Quando NÃO usar: escrever qualquer outro arquivo de `/docs` (→ tech-docs-agent) · escrever ADR (→ system-architect, apenas referencia) · escrever `/product` (→ product-docs-agent) · opinar sobre a qualidade da decisão registrada. **Ferramentas e conhecimentos:** Read, Grep, Glob, Write (`docs/HISTORY.md` apenas); registro factual e neutro. **Exemplos de uso:**

1. *"Registra a fusão dos dois agentes de inteligência de notícia"* → lê o resultado final do trabalho → entrada com data, o que mudou, por quê, alternativas descartadas.
2. *"Registra a correção do bug de dedup"* → confirma que a correção foi de fato mergeada → entrada com arquivos tocados e impacto (ou "a medir").
3. *"Uma fonte foi desativada por reprovação de compliance"* → registra o motivo real, não genérico → entrada citando o risco identificado. **Critérios de sucesso:** toda entrada com os 6 campos completos · nova entrada sempre no topo · alternativa descartada sempre com motivo. **Critérios de falha:** reescrever entrada antiga em vez de adicionar nova · registrar mudança não concluída · usar linguagem de valor sobre a decisão. **Relação com outros subagents:** Upstream: tech-docs-agent e product-docs-agent (entram antes no fluxo). Downstream: `docs-publisher` (usa o resumo mais recente de `docs/HISTORY.md` para popular o changelog interno do PDF). Conflito: com tech-docs-agent e system-architect sobre `docs/adr` (resolvido, ver sobreposições residuais #8 e #9).

docs-publisher

Objetivo: garante que exista sempre um único PDF navegável, atualizado, consolidando `/product` + `/docs` — o formato que se compartilha com investidor, cliente ou dev novo, sem depender de ninguém lembrar de gerá-lo. **Responsabilidades:** ler `/product/*.md` e `/docs/**/*.md` (+ `/docs/adr/*.md`) na ordem definida; rodar `node scripts/build-docs-pdf.js`; sobrescrever sempre `docs/dist/road-news-documentation.pdf`; manter `docs/dist/.docs-pdf-manifest.json` (hash de origem + changelog interno); remover qualquer PDF solto encontrado fora do caminho fixo. **Quando usar:** último passo da Definition of Done, sempre que `tech-docs-agent` ou `product-docs-agent` tiverem alterado algo · comando `/publish-docs` ou pedido explícito ("gera o PDF da documentação", "atualiza o PDF"). **Quando NÃO usar:** decidir conteúdo técnico ou de produto (→ `tech-docs-agent`/`product-docs-agent`, eu só consolido) · rodar a cada commit pequeno isolado (o hash decide se regenera, não o gatilho) · criar um PDF por feature/data/versão (sempre o mesmo arquivo). **Ferramentas e conhecimentos:** Read, Write, Bash, Glob; `markdown-it` + `puppeteer` (as únicas dependências npm do projeto, isoladas em `scripts/package.json`), geração de PDF via Chromium headless, renderização de Mermaid no navegador. **Exemplos de uso:**

1. *"tech-docs-agent acabou de criar ARCHITECTURE.md"* → hash de `/docs` mudou → regenera o PDF, log confirma 1 arquivo lido a mais e novo número de páginas.
2. *"Roda de novo sem eu ter mudado nada"* → hash idêntico ao manifest anterior → "documentação sem alterações desde a última publicação", nenhum commit de binário novo.
3. *"Achei um PDF antigo solto em /docs/relatorio-2025.pdf"* → deleta na próxima execução, registra a limpeza no changelog interno do PDF. **Critérios de sucesso:** existe exatamente 1 PDF de documentação no repositório · rodar 2x sem mudança não gera novo binário · nenhuma seção inventada para arquivo inexistente em `/product//docs`. **Critérios de falha:** criar `road-news-documentation-v2.pdf` ou com sufixo de data · deixar HTML intermediário versionado · renderizar diagrama Mermaid como bloco de código cru quando a renderização tinha condição de funcionar. **Relação com outros subagents:** Upstream: `tech-docs-agent`, `product-docs-agent` (fonte do conteúdo), `project-historian` (resumo para o changelog interno). Downstream: nenhum — é o fim de toda Definition of Done. Conflito: nenhum residual — é o único agente com `Write` em `docs/dist/**` e em `scripts/build-docs-pdf.js/scripts/package.json`.

ADR 0001 — Arquitetura de Subagents do Road News

Status

Aceito — 2026-08-06

Contexto

O Road News cresceu de um scraper simples para um produto multi-tenant real: coleta de notícias rodoviárias, triagem de eventos críticos por IA, painel com controle de acesso por pacote/quota, e um backoffice de provisionamento de cliente — tudo isso já em produção, não em planejamento. Até esta decisão, todo esse trabalho era feito sem divisão de responsabilidade formalizada entre domínios (scraping, IA, dados, segurança, produto, documentação), o que cria dois riscos concretos à medida que o volume de mudança cresce:

1. **Ambiguidade de escopo.** Sem fronteira explícita, a mesma lógica (ex.: extração de rodovia/km, classificação de severidade) tende a ser reimplementada em mais de um lugar — o próprio código já mostra sinais disso (extração de entidade duplicada entre `scraper/utils/image_summary.py` e `api/news-feed.js`).
2. **Perda de contexto de domínio.** Regras finas de negócio (ex.: por que `MIN_CONFIDENCE = 0.82`, por que `SCRAPER_AI_SUMMARIES_ENABLED` está desligado em produção) vivem só na cabeça de quem as escreveu, sem registro que sobreviva à pessoa.

Adicionalmente, a especificação inicial deste roster de agentes foi escrita a partir de uma "stack de referência" genérica (Next.js/NestJS, Playwright/Crawl4AI/Firecrawl, Docker/Coolify/Railway/Azure, Jest/Vitest) que diverge do estado real do repositório: scraper Python sem browser headless, API em Vercel Serverless Functions com Node vanilla e zero dependências (sem `package.json`), frontend em arquivo único sem build step, deploy só Vercel + GitHub Actions, testes com `unittest`/`node:test` nativos. Essa divergência foi confirmada por inspeção direta do código antes de qualquer agente ser escrito (ver `git log`, `requirements.txt`, ausência de `package.json`, `public/index.html` com ~6.800 linhas).

Decisão

Adotamos um roster de **22 subagents em 4 camadas**, documentado em `docs/AGENTS.md` e materializado em `.claude/agents/*.md`:

- **Camada A (13 agentes)** — arquitetura e implementação, com `Write`/`Edit` restrito ao(s) artefato(s) de sua propriedade exclusiva.
- **Camada B (3 agentes)** — governança e revisão, somente leitura de código (`security-reviewer`, `compliance-advisor`, `finops-agent`); nunca implementam, apenas emitem achados priorizados ou decisões quantificadas.
- **Camada C (3 agentes)** — produto e dados (`ux-product-designer`, `data-analyst`, `seo-content-agent`), com escrita restrita a specs/relatórios, não a código de aplicação (exceto `seo-content-agent` em artefatos públicos que não colidem com a aplicação).
- **Camada D (3 agentes)** — documentação e memória institucional, cada um dono de uma fatia disjunta de `/docs`, `/product` e `docs/HISTORY.md`.

Cada agente é definido por: `name`, `description` roteável em terceira pessoa com gatilho de proatividade, `tools` sob o princípio de menor privilégio, `model` (opus para decisão de alto risco/domínio, sonnet para implementação, haiku para registro mecânico de alto volume), e um system prompt estruturado em Identidade / Contexto do produto / Escopo / Como você trabalha / Padrões inegociáveis / Formato de saída / Limites e escalonamento.

Decisão estrutural adicional: os agentes descrevem e defendem a stack real do repositório como linha de base. A stack de referência mais ampla (framework web, headless browser, containerização, runner de teste externo) é tratada como caixa de ferramentas futura, habilitada por `system-architect` via ADR quando um caso concreto — não uma preferência estética — justificar. Essa escolha foi validada explicitamente com o usuário antes da implementação (ver histórico da sessão), diante de duas alternativas: ancorar na stack real ou escrever os agentes já mirando a stack de referência como direção ativa de migração.

Toda mudança segue uma **Definition of Done** obrigatória (detalhada em `CLAUDE.md`): `tech-docs-agent` (toda mudança técnica), `product-docs-agent` (quando altera capacidade percebida pelo cliente, preço ou posicionamento — decide por si

mesmo quando não há impacto), e `project-historian` (toda mudança, sempre por último).

Alternativas consideradas

1. **Agente único generalista.** Rejeitada: um único agente com contexto de scraping, IA, banco, segurança, produto e documentação ao mesmo tempo dilui o contexto disponível para qualquer decisão específica (viola o princípio de contexto mínimo) e não permite `tools` com menor privilégio — um agente que só deveria revisar ganharia `Write` por conveniência.
2. **Roster amplo sem camadas, todos com as mesmas permissões.** Rejeitada: colapsar Camada B (governança) na Camada A removeria a garantia estrutural de que revisão de segurança/compliance/custo nunca é feita pelo mesmo agente que implementou — o viés de quem constrói revisar o próprio trabalho é exatamente o que a separação evita.
3. **Roster técnico apenas (só Camada A), sem produto/documentação como agentes formais.** Rejeitada: já observamos no histórico do projeto que documentação e contexto de negócio decaem mais rápido que código quando não têm dono — a ausência de `/product` e de um `docs/HISTORY.md` estruturado hoje é evidência direta disso.
4. **Espelhar 1:1 a especificação original do roster, sem resolver sobreposições.** Rejeitada: a especificação original continha sobreposições reais (`ADR` reivindicado por três agentes diferentes; `api/**` dividido de forma genérica demais para o código real; landing e painel logado tratados como arquivos separados quando são o mesmo arquivo). Implementar sem resolver essas sobreposições teria produzido exatamente o defeito que o roster deveria evitar: dois agentes com `Write` no mesmo artefato.
5. **Escrever os agentes mirando a stack de referência (Next.js/NestJS/Playwright/Docker/Jest) como direção ativa.** Rejeitada nesta rodada, por escolha explícita do usuário: os agentes ficariam recomendando comandos, arquivos e padrões que não existem no repositório real, o que é mais custoso de corrigir depois do que de ancorar corretamente agora. Permanece disponível como evolução deliberada via ADR futuro.

Sobreposições resolvidas

Sete sobreposições já vinham identificadas na especificação original do roster (fusão dos dois agentes de inteligência de notícia em `logistics-intelligence`; separação `ai-engineer×prompt-engineer`; ampliação de `cost-optimization-agent` para `finops-agent`; eliminação da duplicata `documentation-writer=tech-docs-agent`; hierarquia `product-docs-agent→seo-content-agent`; separação `security-reviewer×compliance-advisor`; separação `data-analyst×`quem corrige). Durante a implementação, identificamos e resolvemos mais sete sobreposições residuais não previstas na especificação original — a mais relevante sendo a divisão de `docs/adr` entre três agentes diferentes na redação original (`system-architect`, `tech-docs-agent` e `project-historian` reivindicavam o mesmo caminho), resolvida atribuindo `Write` exclusivo em `docs/adr/**` só a `system-architect`. A lista completa, com a justificativa de cada resolução, está em `docs/AGENTS.md`, seção "Sobreposições resolvidas".

Consequências

Positivas:

- Nenhum caminho do repositório tem dois agentes com permissão de escrita — verificável diretamente pela matriz de propriedade em `docs/AGENTS.md`.
- Agentes consultivos (Camada B) estruturalmente não podem "corrigir" o que revisam — preserva a separação entre quem implementa e quem audita.
- A régua de severidade e a taxonomia de domínio (hoje fragmentadas entre `event_intelligence.py` e `riskForItem` em `news-feed.js`) passam a ter um dono único (`logistics-intelligence`) responsável por fazer convergir as duas implementações.
- Documentação técnica, de produto e histórico institucional deixam de depender de alguém lembrar de escrevê-las — viram parte obrigatória da Definition of Done.

Custos/trade-offs:

- 22 agentes é mais overhead de manutenção do próprio roster do que um número menor — mitigado pelo teste de responsabilidade exclusiva aplicado a cada um antes de ser incluído (nenhum agente foi criado sem um conjunto de tarefas que só ele cobre).
- A divisão fina de `api/**` entre `backend-engineer` e `saas-platform-engineer`, e de `scraper/weekly_news.py` entre três agentes por sub-responsabilidade, exige que quem orquestra (o agente principal) leia a matriz de propriedade com atenção em vez de assumir "um arquivo, um agente" universalmente — documentado explicitamente para não virar fonte de confusão.
- Ancorar na stack real significa que, se o produto de fato crescer na direção da stack de referência, haverá trabalho de revisão dos agentes técnicos (não do roster em si) nesse momento — aceito como custo futuro menor do que o custo presente de documentar capacidades inexistentes.

Changelog de publicação do PDF

Histórico de quando este PDF foi regenerado e o que mudou desde a publicação anterior — não confundir com o CHANGELOG.md do produto.

Data	O que mudou
2026-08-06	Publicação inicial da documentação consolidada.